

Dossier E5-bis — Kernel Ternari su CPU + Sensibilità Quantizzazione SSM

Estrazione dalla letteratura per motore C su Zen 2 (AVX2, no VNNI) con MLP ternario pshufb-LUT

Brief n.2 — da incollare al Ricercatore

Phase 62 — Design Gate (Cat-A)

Setup: SSM stile Mamba-1 + SWA + MLP gated-dReLU ternario $\{-1,0,+1\}$

Kernel pshufb-LUT: 4.2-5× vs fp32 (bit-exact, misurato)

Proiezioni SSM ternarizzate: +0.018-0.022 BPB (misurato)

Data: 2026-07-04

Sintesi fattuale dei findings

Questo dossier estrae dalla letteratura recente (preprint ultimi 18 mesi + paper peer-reviewed fondativi) i dati pubblicati su: (1) kernel LUT low-bit su CPU, (2) sensibilità alla quantizzazione di SSM/Mamba, (3) training ternario QAT a piccola scala. Tutti i numeri sono etichettati **[MISURATO]** o **[PROIETTATO]** e associati alla rispettiva baseline. **Nota BPB**: bits-per-byte è metrica byte-level di perplexity; i paper citati (BitNet, Quamba, Spectra) riportano tipicamente perplexity o zero-shot accuracy — la conversione richiede assunzioni sul tokenizer (1 BPB $\approx \times 1.44$ PPL factor su test set standard, caveat). Non contiene raccomandazioni — la decisione di design è a valle, con i dati Cat-A di Phase 62.

Punti chiave (solo fatti, no raccomandazioni)

- **Kernel LUT su CPU: il paper più vicino per hardware è DeepGEMM** (CVPR 2023W, Intel i7-9700K Coffee Lake, AVX2-only no VNNI come il nostro Zen 2). $1.74\times$ vs INT8 QNNPACK su W2A2 CNN [MISURATO] → convertito $\approx 7\times$ vs FP32 teorico, **coerente o superiore** al nostro $4.2\text{--}5\times$ vs fp32. bitnet.cpp (ACL 2025) testa pshufb-LUT ternary lossless su Raptor Lake (con AVX-VNNI): range x86 [$2.37\times$, $6.17\times$] vs FP16 contiene il nostro $4.2\text{--}5\times$ — **allineato**. T-MAC README avverte esplicitamente che old-gen x86 non è garantito per speedup: Zen 2 (2019) è old-gen → il nostro $4.2\text{--}5\times$ è **evidenza diretta non coperta** dalla letteratura.
- **Sensibilità SSM: contraddizione parziale con KL-Lens (CVPR 2026W)**. KL-Lens isola esplicitamente **x_proj** come colpevole primario ($\Delta=0.27$, $>10\times$ gli altri sottotipi) → **[CONTRADDIZIONE]** con il nostro finding "sensibilità NON isolata in x/dt" sul lato xproj. *Ma KL-Lens isola anche **dtproj come quasi-insensibile ($\Delta=3.6\times 10^{-4}$) → [ALLINEATO]** sul lato dtproj. Mamba-PTQ (arXiv 2407.12397) conferma: dtproj ha "almost no outliers". MambaQuant (ICLR 2025) non isola x/dt. Quamba/Quamba2 parlano di "x" come tensore SSM input, non come proiezione x_proj — etichetta ambigua.*
- **Trend scala QAT ternary: COSTO CALA al crescere della scala** (convergenza forte). BitNet b1.58: match FP16 a 3B+. Spectra/TriLM (ICLR 2025 Spotlight): gap misurabile sotto 1B, gap → 0 sopra 1B per commonsense/knowledge. BitNet v2: gap 4-bit vs 8-bit activation cala da 400M a 7B. TWN (CNN): gap ResNet18 → ResNet18B cala da -3.6% a -2.3% . **Anchor cliente $+0.013\pm 0.005$ BPB a 5M è consistente** come direzione (gap piccolo ma >0 sotto 99M). PT²-LLM (PTQ ternary) mostra gap $\sim 2\times$ PPL che NON si chiude con scala → conferma indiretta che il trend positivo è specifico QAT, non PTQ.
- **Scala 5M: GAP TOTALE nella letteratura**. Nessun paper ternary QAT scende sotto 99M (Spectra/TriLM è il minimo, $20\times$ più grande del nostro setup). Nessun paper riporta BPB byte-level (tutti PPL o accuracy). Nessun paper usa dReLU gated MLP esplicito (Spectra/TriLM usa SwiGLU, affine ma non identico). Nessun paper QAT ternary su SSM. Nessun paper QAT ternary su single-user CPU.

- **Nessuna contraddizione formale con i nostri numeri.** Speedup pshufb-LUT 4.2-5× vs fp32 su Zen 2 è allineato al range x86 dichiarato di bitnet.cpp [2.37×, 6.17×] e coerente con DeepGEMM su hardware simile. Costo ternary $+0.013 \pm 0.005$ BPB a 5M è consistente con l'estrapolazione del trend QAT (gap piccolo ma >0 sotto 99M). Contraddizione parziale solo su KL-Lens (x_proj isolato come colpevole) — da dichiarare esplicitamente in review.

1. Domanda 1 — Kernel LUT low-bit su CPU

Otto lavori primari identificati + 1 menzione secondaria. Coprono: T-MAC (Microsoft, EuroSys 2025), bitnet.cpp (Microsoft, ACL 2025), BitNet originale (framing QAT), LUT-NN (MobiCom 2023, centroid-based), ShiftAddLLM (NeurIPS 2024, GPU), TernGEMM (SiPS 2021, add/sub-based), DeepGEMM (CVPR 2023W, AVX2 LUT), T-SAR (nov 2025, ISA extension gem5 sim), TABv2 (IEEE TVLSI 2026, popcount). **Nessun paper testa su Zen 2 specifico** — DeepGEMM su Intel i7-9700K Coffee Lake (AVX2-only no VNNI) è l'hardware più vicino.

1.1 T-MAC — CPU Renaissance via Table Lookup

Citazione: [1] Wei, J.; Cao, S.; Cao, T.; Ma, L.; Wang, L.; Zhang, Y.; Yang, M. (USTC; Microsoft Research; UCAS), *T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge*, EuroSys 2025; arXiv:2407.00088. URL: <https://arxiv.org/abs/2407.00088> — repo: <https://github.com/microsoft/T-MAC>.

Tecnica esatta. LUT su **pesi low-bit** (int1/2/3/4) moltiplicati per attivazioni high-precision (int8/fp16/fp32). Le attivazioni **non** sono quantizzate oltre la precisione nativa (es. W1A8, W2A16, W4A16). Ogni peso è decomposto bit-wise in b matrici a 1 bit. Per ogni gruppo di g elementi di un bit-plane ($g=4 \rightarrow 2^4 = 16$ pattern), si precomputano tutte le combinazioni activation·pattern e si salvano in una LUT di 2^g entries. L'MAC si riduce a (lookup + add). Istruzioni SIMD: **TBL su ARM NEON** (vqtbl1q_u8), **PSHUF su x86 AVX2** (_mm256_shuffle_epi8) — entrambe 8-bit lookup. ARM NEON 128-bit ospita intera LUT $g=4$ (16 byte). Intel AVX2 256-bit ha due lane 128-bit separate $\rightarrow$ T-MAC **duplica la LUT** per riempire i 256 bit e look-up di 32 indici int8 con una sola istruzione. Bit-packing: pesi impaccati come bit-planes (1 bit-plane per ogni bit del peso). Group size $g=4$. Tecniche di riduzione memoria: **mirror consolidation** (metà entries inversa dell'altra $\rightarrow$ dimezza) e **table quantization** (fp16 $\rightarrow$ int8 con scaling).

Speedup misurato [MISURATO], baseline = llama.cpp (kernel dequantization-based). BitNet-b1.58-3B throughput: 30 tok/s single-core, 71 tok/s 8-core su Apple M2-Ultra; 11 tok/s su Raspberry Pi 5. Kernel-level (mpGEMV single-thread, matrici Llama-2-7B/13B): max **11.2×** (1-bit), 5.8× (2-bit), 4.7× (3-bit), 3.1× (4-bit). Kernel-level (mpGEMV multi-thread, 2-bit): 4.0×, 4.0×, 5.31×, 2.5× (M2-Ultra, Orin, Surface, RBP). Kernel-level (mpGEMM multi-thread, seq=256, 2-bit): 4.0× (RBP), 5.3× (Orin), 5.3× (Surface); 2.0× su M2-Ultra per 1-bit (Apple AMX limita). E2E: 2.8× per Llama-2-7B-2bit; 2.8×/6.7×/5.8× single-thread RBP su tre modelli; 1.1×/2.3×/1.7× multi-thread M2-Ultra. Abstract headline: "up to 4× increase in throughput and 70% reduction in energy". Throughput assoluto su Surface Laptop 7 (Snapdragon X Elite, ARM): 20 tok/s single-core, 48 tok/s 4-core per

3B BitNet. Throughput assoluto Snapdragon X Elite vs NPU: 12.6 tok/s 2-core, 22 tok/s multi-core CPU T-MAC vs 10.4 tok/s NPU (llama-2-7b-4bit). Energy: 60-70% riduzione vs llama.cpp.

Hardware. CPU testate: Apple M2-Ultra (ARM, AMX coprocessor), Jetson AGX Orin (ARM Cortex-A78AE), **Surface Book 3 — Intel Core i5-1035G7 (Ice Lake, AVX-512 disponibile ma paper usa solo path AVX2)**, Raspberry Pi 5 (ARM Cortex-A76). Surface Laptop 7 (Snapdragon X Elite, ARM). ISA: NEON (ARM), **AVX2 (x86)**. NON usa AVX-512, NON usa VNNI, NON usa AMX. **README esplicito:** *"We have noticed many users attempting to evaluate T-MAC on old-gen x86 platforms. However, x86 CPUs vary dramatically, and due to unawareness of AI workloads, most of these platforms have extremely low memory bandwidth (even lower than Raspberry Pi 5). Our current tests do not encompass all x86 platforms, particularly older generations. We cannot guarantee significant speedup (especially for 4-bit token generation) on all x86 platforms. We recommend Surface Book 3 or ARM devices to evaluate T-MAC."*

Regime. mpGEMV multi-thread dichiarato **memory-bandwidth-bound** ("T-MAC's performance is primarily constrained by memory bandwidth"). mpGEMV single-thread: compute-bound (sfrutta LUT in registri). Batch size: single-user (token generation decode, batch=1) è caso principale; mpGEMM batch>1 (seq=256) per prefill. Multi-thread: testato fino a 8 core (M2-Ultra) e 12 core (Orin).

Cosa copre del nostro caso. pshufb-LUT su AVX2: **SÌ** (path esplicito, `_mm256_shuffle_epi8` con duplicazione LUT). Ternary $\{-1,0,+1\}$: **SÌ** (W1.58 via bit-wise decomposition; testato su BitNet-b1.58-3B). Single-user: **SÌ** (decode batch=1 è il caso headline). Bit-exact: **NO** — T-MAC usa table quantization (fp16 → int8) per ridurre memoria, introdurrebbe perdita.

Cosa NON copre. Zen 2 specifico: **NO** — non testato. Il README avverte esplicitamente contro *old-gen* x86. Zen 2 (2019) è *old-gen*. Bit-exact: **NO** (table quantization introduce perdita). MLP ternary con gated-dReLU: **NO** (testato su BitNet b1.58, architettura SwiGLU/standard, non dReLU). BPB byte-level: **NO** (misura throughput in tok/s). SSM selettivo: **NO** (solo Transformer).

Contraddizioni con le nostre misure. T-MAC riporta $5.8\times$ (2-bit single-thread GEMV) vs llama.cpp su Surface Book 3 — **superiore al nostro $4.2-5\times$ vs fp32**, ma baseline diversa (llama.cpp dequant non fp32 puro, hardware diverso Ice Lake vs Zen 2). Su 4-bit scende a $3.1\times$ — **inferiore al nostro range $4.2-5\times$** (ma noi siamo 1.58-bit ternary, non 4-bit; confronto non diretto). README esplicito su *old-gen* x86: **non garantisce speedup su Zen 2**. La nostra misura $4.2-5\times$ su Zen 2 è quindi evidenza nuova non coperta.

1.2 bitnet.cpp — Efficient Edge Inference for Ternary LLMs

Citazione: [2] Mo, Z.; Wang, L.; Wei, J.; Zeng, Z.; Cao, S.; Ma, L.; Jing, N.; Cao, T.; Xue, J.; Yang, F.; Yang, M. (SJTU; Microsoft Research; USTC), *Bitnet.cpp: Efficient Edge Inference for Ternary LLMs*, ACL 2025 (Long); arXiv:2502.11880. URL: <https://arxiv.org/abs/2502.11880> — repo: <https://github.com/microsoft/BitNet>. Precursore: *1-bit AI Infra: Part 1.1, Fast and Lossless BitNet b1.58 Inference on CPUs*, ottobre 2024.

Tecnica esatta. LUT su **pesi ternari** $\{-1,0,+1\}$ con attivazioni int8 (BitNet b1.58 schema training-time: ternary weights + per-tensor int8 activations). Due soluzioni core: **TL (Ternary Lookup**

Table) e I2_S (Int2 with Scale). TL = element-wise LUT (ELUT): invece di bit-wise decomposition (T-MAC), enumera direttamente C^g possibili valori ($C=3$ per ternary) per gruppo di g pesi. $g=2 \rightarrow$ TL1 (4 bit per 2 pesi, $3^2 < 2^4$). $g=3 \rightarrow$ TL2 (5 bit per 3 pesi con mirror consolidation, $3^3/2 < 2^4$), $\text{bpw} \approx 1.67$. I2S = MAD-based, 2 bit per peso + scaling, lossless per BitNet b1.58 (mantiene settings di training: ternary weights + per-tensor int8 activations, K multiplo di 128). Istruzioni SIMD: ****AVX2 `vpshufb` (=mm256shufflepi8) + AVX2 `\_mm256\_maddubs\_epi16` per I2_S; NEON vqtbl1q_u8 per TL su ARM. Lookup width 128 bit $\rightarrow$ max 16 int8 entries per lookup $\rightarrow$ limita $g \leq 3$. Per lossless: pack-and-unpack** technique — somme mantenute int16, split in due 8-bit LUT (low/high), lookup due volte, unpack a int16. Kernels TL11/TL21 = lossless; TL10/TL20 = con quantizzazione addizionale (non lossless).**

Speedup misurato [MISURATO], baseline = llama.cpp FP16 (lossless reference) + llama.cpp low-bit (TQ1_0, TQ2_0) + T-MAC. $6.25\times$ vs FP16 (I2S, headline). $2.32\times$ vs T-MAC (TL20 su Intel i7-13700H); $1.19\times$ su Apple M2 Ultra. $1.33\times$ vs TQ10 (Intel i7-13700H); $1.65\times$ su Apple M2 Ultra. 100B ternary LLM: TL20 raggiunge 7.45 tok/s su Apple M2 Ultra, 1.69 tok/s su Intel i7-13700H. Precursore Microsoft Part 1.1 (ott 2024): $2.37\times$ – $6.17\times$ on x86 CPUs, $1.37\times$ – $5.07\times$ on ARM CPUs, energy reduction 71.9-82.2% (x86) / 55.4-70.0% (ARM). 100B BitNet b1.58 su singola CPU: 5-7 tok/s (human reading speed). Microbenchmark istruzioni (Intel i5-13400F): singola MAD = 3.77 ns, singola TBL = 3.70 ns (stesso throughput); TBL+ADD+CVT = 6.20 ns (+68% vs MAD singolo). Throughput assoluto GB/s: Apple M2 Ultra >800 GB/s max bandwidth; Intel i7-13700H <100 GB/s max bandwidth (PCM misurato). Saturazione bandwidth a 4 thread su i7-13700H.

Hardware. CPU testate (paper ACL 2025): **Apple M2 Ultra (ARM, >800 GB/s)** e **Intel i7-13700H (x86, Raptor Lake, AVX2 + AVX-VNNI, <100 GB/s)**. Microbenchmark aggiuntivo: **Intel i5-13400F (Raptor Lake, AVX2 + AVX-VNNI)**. ISA: AVX2 (x86), NEON (ARM). NON usa AVX-512, NON usa AMX. (Raptor Lake ha AVX-VNNI ma il paper usa AVX2 path.) **NON testato su Zen 2 / AMD.** NON testato su hardware senza VNNI esplicito (anche se il path AVX2 non usa VNNI).

Regime. Decode (single-user, GEMV batch=1): **memory-bandwidth-bound**. Paper esplicito: "When the thread count reaches 4, the token throughput also reaches its maximum value due to the saturation of the bandwidth, causing the end-to-end inference speed to reach its peak. Therefore, we can conclude that the maximum bandwidth limits the potential of ELUT." Prefill (batch>1, GEMM): compute-bound. Multi-thread testato fino a saturazione bandwidth. Batch size: decode batch=1 (single-user); prefill seq=128/256.

Cosa copre del nostro caso. pshufb-LUT su AVX2: **SÌ** (`_mm256_shuffle_epi8`, lookup 128-bit in lane 256-bit). Ternary $\{-1,0,+1\}$: **SÌ** (TL1/TL2 esplicitamente ternary). Bit-exact: **SÌ** (TL11/TL21 + I2_S sono lossless vs BitNet b1.58 training). Single-user: **SÌ** (decode batch=1 principale).

Cosa NON copre. Zen 2 specifico: **NO**. Testato su Raptor Lake (i7-13700H, 2023) e Apple M2 Ultra. Zen 2 (2019) è 4 generazioni prima. MLP gated-dReLU: **NO** (testato su BitNet b1.58 SwiGLU standard). SSM: **NO**. BPB byte-level: **NO** (tok/s).

Contraddizioni con le nostre misure. bitnet.cpp headline **6.25× vs FP16** lossless — leggermente **superiore** al nostro 4.2-5× vs fp32. Ma: (a) hardware diverso (Raptor Lake con bandwidth <100 GB/s ma con AVX-VNNI e cache hierarchy più recente di Zen 2), (b) modello diverso (BitNet b1.58 standard vs nostro MLP gated-dReLU). 2.37× è il **lower bound** su x86 — sotto al nostro 4.2×. Range paper [2.37×, 6.17×] su x86 contiene il nostro valore 4.2-5×. **Allineato**, non contraddittorio. Nessun test diretto su Zen 2 → il nostro numero è evidenza diretta non coperta.

1.3 BitNet originale — framing QAT (non kernel)

Citazione: [3] Wang, H.; Ma, S.; Dong, L.; Huang, S.; Wang, H.; Ma, L.; Yang, F.; Wang, R.; Wu, Y.; Wei, F. (Microsoft Research Asia + UWashington + USTC + CAS), *BitNet: Scaling 1-bit Transformers for Large Language Models*, arXiv:2310.11453; JMLR vol. 26 (24-2050), 2025. URL: <https://arxiv.org/abs/2310.11453>.

Tecnica esatta. Non è un kernel CPU. È il paper QAT originale che introduce **BitLinear** come drop-in replacement di nn.Linear, con pesi binari $\{-1, +1\}$ (1-bit, non ternary) e attivazioni quantizzate a 8-bit absmean. LUT/pshufb: non applicabile (training-side, non inference kernel). Bit-packing: 1 bit per peso. **Speedup:** non misura kernel CPU diretto; inference benefits dichiarati in termini di memory footprint + consumo energetico vs FP16 e INT8 quant (paper JMLR riporta energy reductions teoriche da bit-line arithmetic vs FP16 moltiplicazioni). **Hardware:** training su GPU; inference non testato su CPU nel paper originale. **Cosa copre:** framing concettuale (ternary/binary QAT, BitLinear). **Cosa NON copre:** kernel CPU, pshufb-LUT, Zen 2, single-user, bit-exact, SSM, BPB, MLP gated-dReLU. **Contraddizioni:** nessuna (paper di framing, non riporta kernel speedup).

1.4 LUT-NN — Centroid Learning + Table Lookup

Citazione: [4] Tang, X.; Wang, Y.; Cao, T.; Zhang, L.L.; Chen, Q.; Cai, D.; Liu, Y.; Yang, M. (Microsoft Research + Zhejiang U.), *LUT-NN: Empower Efficient Neural Network Inference with Centroid Learning and Table Lookup*, MobiCom 2023; arXiv:2302.03213. URL: <https://arxiv.org/abs/2302.03213>.

Tecnica esatta. LUT su centroidi: NON è bit-wise decomposition. LUT-NN impara *centroidi* (valori tipici) per ogni operatore via differentiable centroid learning (backprop). Precomputa risultati per ogni centroide e li salva in LUT. A inference, l'input closest-centroid cerca e lookup. Istruzioni SIMD: non specificate in abstract; paper dice "*implemented for Intel and ARM CPUs*" con "*dedicated hardware units for optimal performance*". Bit-packing: non specificato (centroid-based, non bit-wise).

Speedup misurato [MISURATO], baseline = related work. Riduzioni: FLOPs $\leq 16\times$, model size $\leq 7\times$, **latency** $\leq 6.8\times$, memory $\leq 6.5\times$, power $\leq 41.7\%$. Accuracy: 66-92% improvement vs related work (similar a originali). Numeri esatti su matrici M/N/K: non in abstract. **Hardware:** CPU Intel + ARM (non specificato modello in abstract). **Regime:** on-device DNN inference (CNN, speech, NLP). Single-user edge.

Cosa copre. LUT su CPU: **SÌ** (genericamente). Single-user edge: **SÌ**. **Cosa NON copre.** pshufb-LUT bit-wise: **NO** (centroid-based, diverso paradigma). Ternary $\{-1,0,+1\}$: **NO** (centroid quantization, non ternary puro). Bit-exact: **NO** (approximation by centroids). Zen 2: **NO**. MLP ternary: **NO** (testato su CNN/speech/NLP generici, non LLM ternary). SSM: **NO**. BPB: **NO**. **Contraddizioni:** LUT-NN riporta latency $\leq 6.8\times$ — **superiore** al nostro $4.2\text{-}5\times$ ma su workload diverso (CNN, non LLM MLP ternary) e con perdita di accuracy. Non confrontabile direttamente.

1.5 ShiftAddLLM — Multiplication-Less Reparameterization (GPU)

Citazione: [5] (Georgia Tech; Intel Labs; Google; Google DeepMind), *ShiftAddLLM: Accelerating Pretrained LLMs via Post-Training Multiplication-Less Reparameterization*, NeurIPS 2024; arXiv:2406.05981. URL: <https://arxiv.org/abs/2406.05981> — repo: <https://github.com/GATECH-EIC/ShiftAddLLM>.

Tecnica esatta. LUT su **pesi binarizzati via BCQ (Binary-Coding Quantization)**: ogni peso matrice $\rightarrow$ binary matrices + group-wise power-of-two scaling factors. Moltiplicazione sostituita con (1) shift tra attivazioni e scaling factors PoT, (2) **LUT queries** per ogni 8 elementi ($2^8 = 256$ entries precomputed) + add. LUT su attivazioni: le attivazioni restano **FP16** (weight-only quantization + LUT). Istruzioni SIMD: **CUDA kernels su GPU** (custom BCQ kernel basato su `lut_gemm`). **NON CPU**. Bit-packing: BCQ format, binary matrices + $\text{sign} \cdot 2^P$ scaling.

Speedup misurato [MISURATO], baseline = OPTQ, LUT-GEMM, QuIP, AWQ. Hardware: **single A100-80GB GPU (PCIe)**. Accuracy: -5.6 PPL (3-bit) e -22.7 PPL (2-bit) avg vs best baseline; latency reduction 6.5-60.1%; energy savings 26-89.9%. Memory: 80-87% riduzione vs FP16 (OPT-66B: 23GB vs 122GB at 3-bit; LLaMA-2-70B: 25GB vs 128GB). Energy stimato con Eyeriss-like model (non misurato diretto): 87.2% reduction OPT-66B, 86.0% LLaMA-2-70B. **Hardware:** GPU NVIDIA A100-80GB (PCIe). ISA: CUDA. **NON CPU**. **NON AVX2**.

Cosa copre. LUT su pesi binary con attivazioni high-precision: **SÌ** (stesso principio del nostro setup). Concetto di sostituire moltiplicazione con LUT+shift+add: **SÌ**. **Cosa NON copre.** CPU: **NO** (GPU only). pshufb: **NO**. AVX2: **NO**. Zen 2: **NO**. Ternary $\{-1,0,+1\}$: **NO** (binary BCQ, non ternary). Bit-exact: **NO** (post-training approximation). SSM: **NO**. BPB: **NO**. **Contraddizioni:** nessuna diretta (GPU workload, baseline e metriche diverse). ShiftAddLLM è solo referenza concettuale per LUT-based multiplication-less inference; non comparabile al nostro kernel CPU.

1.6 TernGEMM — Ternary Weights GEMM (add/sub-based)

Citazione: [6] Choi, S.; Shim, K.; Choi, J.; Sung, W.; Shim, B. (Seoul National University), *TernGEMM: General Matrix Multiply Library with Ternary Weights for Fast DNN Inference*, IEEE SiPS 2021. DOI: <https://ieeexplore.ieee.org/document/9605039> — repo: <https://github.com/aiha-lab/TernGEMM>.

Tecnica esatta. LUT su **pesi ternari** $\{-1,0,+1\}$ con attivazioni sotto 8-bit. **NO MUL instructions** (TernMAC): moltiplicazione con $\{-1,0,1\}$ non cambia bit precision $\rightarrow$ basta add/sub. Implementazione: **assembly kernels** per ARM e Intel CPUs, 4 varianti di tiling (1×96 , 1×128 , 2×64 , 4×32). Istruzioni SIMD: non specificate nel README (assembly inline). **NON esplicitamente**

pshufb-LUT; piuttosto **add/sub-based** su pesi ternari packed. Bit-packing: pesi ternari packed (2 bit per peso); attivazioni sub-8-bit.

Speedup misurato [MISURATO], baseline = GEMMLowp (Google INT8 GEMM library). Kernel-level: **1.68× speedup vs GEMMLowp** (25% meno istruzioni, IPC 2.91 vs 2.16 → +35% IPC). DNN-level: ResNet50 avg 2.052×, MobileNet-V2 2.973×, EfficientNet-B0 2.986× vs GEMMLowp. Tile flexibility: addizionale +18.96% da tile size tuning. Numeri esatti GEMM M/N/K: non nel README. Throughput TFLOP/s o GB/s: non riportato. **Hardware:** CPU Intel + ARM (non specificato modello). ISA: SIMD (assembly), presumibilmente AVX2 (Intel) + NEON (ARM) dato l'anno (2021). **Regime:** DNN inference (CNN, non LLM). Single-thread kernel benchmark (1000 runs averaged). Batch size: non specificato (CNN inference, batch=1 tipico).

Cosa copre. Ternary $\{-1,0,+1\}$ su CPU SIMD: **SÌ**. Single-user edge: **SÌ**. **Cosa NON copre.** pshufb-LUT: **NO** (add/sub-based, non pshufb-LUT). Bit-exact: **SÌ** (TernMAC è bit-exact per costruzione, ma non esplicitato). Zen 2: **NO** (hardware non specificato). LLM: **NO** (CNN, ResNet50/MobileNet/EfficientNet). SSM: **NO**. BPB: **NO**. MLP gated-dReLU: **NO**. Baseline vs INT8 (GEMMLowp), NON vs FP32 — confronto con nostro 4.2-5× vs fp32 non diretto. **Contraddizioni:** TernGEMM 1.68× vs INT8 QNNPACK — **inferiore** al nostro 4.2-5× vs fp32, ma baseline diversa (INT8 vs FP32 $\approx 4\times$ aggiuntivo teorico). Convertito: 1.68× vs INT8 $\approx 6-7\times$ vs FP32 se INT8 è già $\sim 4\times$ più veloce di FP32 — **coerente o superiore** al nostro range, ma su CNN non LLM. Non contraddittorio.

1.7 DeepGEMM — Ultra Low-Precision LUT on CPU (hardware più vicino)

Citazione: [7] Ganji, D.C.; Ashfaq, S.; Saboori, E.; Sah, S.; Mitra, S.; AskariHemmat, M.; Hoffman, A.; Hassanien, A.; Léonardon, M. (Intel; IIT Hyderabad; altri), *DeepGEMM: Accelerated Ultra Low-Precision Inference on CPU Architectures Using Lookup Tables*, CVPR 2023 Workshops, pp. 4656-4664.

URL:

https://openaccess.thecvf.com/content/CVPR2023W/ECV/html/GanjiDeepGEMMAcceleratedUltraLow-PrecisionInferenceonCPUArchitecturesUsingLookupCVPRW2023_paper.html — repo: <https://github.com/Emily0330/DeepGEMM-paper-reproduce>.

Tecnica esatta. LUT su **pesi 2-bit + attivazioni 2-bit (W2A2)**: precomputa tutti i possibili prodotti (weight, activation) e salva in LUT. LUT-16 = 16 entries per 4 weights $\times$ 4 activations (2-bit each → 4 valori → $4 \times 4 = 16$ combinazioni). Istruzioni SIMD: **AVX2 su x86 con 256-bit vector registers**. LUT fits in L1 cache. Bit-packing: pesi 2-bit packed (4 weights per byte), attivazioni 2-bit packed. Unpacking + lookup + accumulation. Bit-exact: **SÌ** (LUT precompute di tutti i possibili prodotti, nessuna approssimazione).

Speedup misurato [MISURATO], baseline = QNNPACK INT8 kernels. 1.74× speedup vs INT8 QNNPACK su x86 (2-bit DeepGEMM vs 8-bit QNNPACK). Tested on **Intel i7-9700K @ 3.6GHz** (Coffee Lake, 2018, AVX2 no AVX-512, no VNNI). 6 CNNs tested end-to-end (QNNPACK vs DeepGEMM), tutti convolution layers 2-bit. Istruzioni per convolution comparabili a FP32 baseline (DeepGEMM claim). Throughput TFLOP/s o GB/s: non riportato nel CVPR abstract.

Hardware. CPU: **Intel i7-9700K @ 3.6GHz (Coffee Lake, 2018).** AVX2 (no AVX-512, no VNNI). **Hardware closest al nostro Zen 2** (entrambi AVX2-only consumer, no VNNI). ISA: **AVX2 with 256-bit registers.** **Regime:** CNN inference (ImageNet-style). LUT in L1 cache → compute-bound per convolution layer. Batch size: tipico batch=1 inference CNN.

Cosa copre del nostro caso. x86 AVX2-only consumer CPU **senza VNNI: SÌ** (Intel i7-9700K Coffee Lake è AVX2-only, architetturealmente simile a Zen 2). LUT-based su CPU: **SÌ**. Bit-exact: **SÌ** (precompute completo). LUT fits in L1: **SÌ** (rilevante per cache profile Zen 2). **Cosa NON copre.** Ternary $\{-1,0,+1\}$: **NO** (W2A2 uniform 2-bit, non ternary 1.58-bit). pshufb esplicito: **NO** (tecnica LUT-16 generica AVX2; non specifica pshufb vs altri). LLM: **NO** (CNN). SSM: **NO**. MLP gated-dReLU: **NO**. BPB: **NO**. Single-user: **SÌ** (CNN inference batch=1).

Contraddizioni con le nostre misure. DeepGEMM $1.74\times$ vs INT8 QNNPACK — **inferiore** al nostro $4.2\text{--}5\times$ vs FP32. Ma: (a) baseline INT8 non FP32 ($\text{INT8} \approx 4\times$ più veloce di FP32, quindi $1.74\times$ vs $\text{INT8} \approx 7\times$ vs FP32 in teoria); (b) W2A2 (2-bit uniform) non ternary W1.58; (c) CNN workload non LLM. Convertito, è **coerente** con il nostro range, non contraddittorio. **Importante:** DeepGEMM è l'unico paper testato su hardware AVX2-only consumer senza VNNI come il nostro Zen 2. Il fatto che abbiamo ottenuto speedup positivi su i7-9700K **supporta** la fattibilità del nostro setup su Zen 2.

1.8 T-SAR — Full-Stack Co-design CPU Ternary (gem5 simulation)

Citazione: [8] Oh, H.; Nam, K.; Bhattacharjya, R.; Chen, H.; Das, T.; Yun, S.; Jang, S.; Ding, A.; Dutt, N.; Imani, M. (UC Irvine), *T-SAR: A Full-Stack Co-design for CPU-Only Ternary LLM Inference via In-Place SIMD ALU Reorganization*, arXiv:2511.13676, novembre 2025. URL: <https://arxiv.org/html/2511.13676v1>.

Tecnica esatta. LUT su **pesi ternari** $\{-1,0,+1\}$ decomposti in due LUT binarie via **ternary-to-binary decomposition**: $wD \in \{-1,+1\}$ (dense, w_i se $w_i \neq 0$ altrimenti $+1$) e $wS \in \{0,1\}$ (sparse, 1 se $w_i = 0$ altrimenti 0). $y = \sum wD \cdot a - \sum wS \cdot a$. Ogni LUT binaria ha 2^c entries (c = block size). ****Generazione LUT in-register**** (no DRAM): $TLUTc \times s$ instruction + $TGEMV k \times m$ instruction. ****Richiede ISA extension custom**** (non puro software!). Istruzioni SIMD: ****x86 AVX2 (focus)****, estendibile a ARM NEON (con SDOT/UDOT ARMv8.2-A) e RISC-V Vector. Implementato come estensione ISA in gem5 ($TLUT2 \times 4$, $TGEMV_8 \times 16$ su YMM 256-bit). Bit-packing: ternary → dense binary + sparse binary. Total LUT storage per block = 2^{c+1} (vs 3^c naive).

Speedup misurato [SIMULATO in gem5, NON silicio reale], baseline = Bitnet.cpp TL-2 + T-MAC. GEMM latency reduction: **5.6–24.5×**. GEMV throughput improvement: **1.1–86.2×**. Prefill (GEMM, batch=128): geo-mean **8.8× (Workstation Ryzen 9950X)**, **8.4× (Laptop Ryzen 7840U)**, **12.4× (Mobile Intel N250)**. Decode (GEMV, batch=1): $6.4\times$ (Workstation), $4.1\text{--}4.2\times$ (Laptop/Mobile). LLM 7B prefill su Mobile: da $>20s$ a $<1.7s$. Cross-platform (batch=1 decode): T-SAR Workstation 128.96 tok/s su Llama-b1.58-8B (0.616 J/tok); Jetson AGX Orin GPU 16.78 tok/s (1.839 J/tok) → $7.7\times$ throughput, $3.0\times$ energy. Memory request reduction: $8.7\text{--}13.8\times$ vs TL-2.

Hardware. gem5 simulation (NON silicio reale!) su: Workstation AMD Ryzen 9950X (Zen 5, 16 core, 5.7 GHz, DDR5-6400); Laptop AMD Ryzen 7840U (Zen 4, 8 core, 5.1 GHz, DDR5-4400); Mobile Intel Processor N250 (4 core, 3.8 GHz, DDR5-4400). ISA: x86 AVX2 + custom T-SAR

extensions. Hardware overhead sintesi TSMC 28nm: +1.4% area, +3.2% power (256-bit SIMD slice). **NON testato su Zen 2** (Zen 4/Zen 5 sono più recenti; richiede ISA extension che non esiste su Zen 2).

Regime. Decode (GEMV): memory-bandwidth-bound (T-SAR rimuove TLUT traffic → diventa compute-bound). Prefill (GEMM): compute-bound. Batch size: prefill N=128 (batch=1 user, multi-token), decode batch=1 single-user. Multi-thread: {16, 8, 4} per {Workstation, Laptop, Mobile}.

Cosa copre. Ternary {-1,0,+1}: **SÌ**. AVX2 ISA: **SÌ** (focus esplicito). Single-user decode: **SÌ** (batch=1). Bit-exact: **SÌ** (decomposizione esatta, nessuna approssimazione). **Cosa NON copre.** pshufb-LUT puro software: **NO** — richiede **ISA extension custom** ($TLUTc \times s + TGEMV_k \times m$). Non esiste su Zen 2 commerciale. Zen 2: **NO** (Zen 4/Zen 5 simulati). MLP gated-dReLU: **NO** (BitNet b1.58 standard). SSM: **NO**. BPB: **NO**. Misurato su silicio: **NO** (gem5 simulation). **Contraddizioni:** T-SAR 5.6-24.5× GEMM, 1.1-86.2× GEMV — **superiore** al nostro 4.2-5×, ma: (a) [SIMULATO] in gem5 non silicio, (b) richiede ISA extension non disponibile su Zen 2, (c) vs Bitnet.cpp TL-2 baseline (non FP32 puro). Non contraddittorio: il nostro 4.2-5× è puro software su Zen 2 commerciale, T-SAR è co-design hardware-software su hardware che non esiste ancora.

1.9 TABv2 / FasterTNN — Ternary/Binary via Popcount (menzione secondaria)

Citazione: [9] Fu, G.; Fischer, O.; Zhu, S.; Alonso, G. (ETH Zurich, FPGA Systems Group), *TABv2: A Faster Ternary and Binary Neural Network Inference Library on the Edge*, IEEE TVLSI 2026 (DOI: 10.1109/TVLSI.2025.3650684). Precursore: *Faster Ternary and Binary Neural Network Inference on CPU by Reducing Popcount Overhead*, ISLPED 2025. URL: <https://ieeexplore.ieee.org/document/11360778> — repo: <https://github.com/fpgasystems/FasterTNN>.

Tecnica esatta. NON LUT-pshufb. È **bitwise GEMM con popcount**: ternary/binary weights → XNOR + popcount. Innovazione: replace scalar popcount on AVX2 with SIMD popcount, reduce complexity via ternary encoding. Istruzioni SIMD: AVX2 SIMD popcount (non pshufb). **Speedup** dichiarato: faster than baseline ternary/binary bitwise GEMM (numeri esatti nel paper IEEE, paywall) [MISURATO]. **Hardware:** CPU AVX2 (Intel) + ARM NEON. GPU CUDA path separato (TABv2GPU). ****Regime**:** *edge inference, ternary/binary NN*. ****Cosa copre**:** *ternary su AVX2 CPU: **SÌ***. ****Cosa NON copre**:** *pshufb-LUT **NO** (popcount-based, diverso paradigma)*. **Bit-exact:** ****SÌ**** (bitwise è esatto). **LLM:** ****NO**** (CNN/NN generiche). **SSM, BPB, gated-dReLU, Zen 2:** ****NO****. ****Contraddizioni**:** *numeri non estraibili (paywall)*. Citato per completezza del landscape ternary CPU SIMD_ — ma diverso paradigma (popcount vs LUT).

2. Domanda 2 — Quantizzazione di SSM/Mamba: organi sensitive

Cinque paper primari + 1 secondario identificati. Coprono: Quamba (ICLR 2025, PTQ Mamba-1), Quamba2 (ICML 2025, PTQ Mamba-1+Mamba-2), KL-Lens (CVPR 2026W, mixed-precision

sensitivity), Mamba-PTQ (arXiv lug 2024, outlier analysis), MambaQuant (ICLR 2025, rotation-based PTQ), Q-Mamba (ACL Findings 2025, state cache quant). **Convergenza forte:** SSM input x (tensore, non proiezione) è sensibile in Quamba/Quamba2/KL-Lens. **Divergenza sul finding "non isolata in x/dt ":** KL-Lens isola $xproj$ come colpevole primario → contraddizione parziale; Mamba-PTQ conferma $dtproj$ non sensibile → allineato.

2.1 Quamba — PTQ Recipe for Selective SSM

Citazione: [10] Chiang, H.-Y.; Chang, C.; Frumkin, N.; Wu, K.-C.; Marculescu, D. (IST Austria; UT Austin; NYCU), *Quamba: A Post-Training Quantization Recipe for Selective State Space Models*, ICLR 2025; arXiv:2410.13229. URL: <https://arxiv.org/abs/2410.13229> — codice: <https://github.com/enyac-group/Quamba>.

Metodo. PTQ (post-training), static per-tensor quantization. Precisione target: **W8A8** (pesi e attivazioni a 8-bit). Explorati anche W2A16 (Quip#-SSM) e W4A4 (QuaRot-SSM) come baselines — entrambi falliscono su Mamba (Tab.7-8). Su **Mamba-1** (130M, 370M, 1.4B, 2.8B) e **Jamba-52B** (hybrid SSM-attention-MoE). NON Mamba-2. Tecnica: percentile-based activation clamping sull'input x al SSM + Hadamard transform sull'output y del SSM per eliminare outlier.

Organi quantization-sensitive (CRITICO). SSM input activation x = organo identificato come PIÙ sensibile. Citazione testuale: " x is highly sensitive to quantization errors and it leads to the largest errors in the SSM output y , although the values in the x tensor are numerically small (≤ 10)" (§4.2, Appendix B). Sensitivity analysis in Fig.6: quantizzare solo SSM I/O (FP16/I8 o I8/FP16) causa performance severe degradation; modello collassa se tutto a 8-bit senza le tecniche di Quamba. **SSM output y** = organo con outlier massivi (≥ 100); affrontato con Hadamard transform. **Projection layers** ($inproj$, $outproj$, $xproj$, $dtproj$): trattate come *dense matrix multiplications*, quantizzate a 8-bit con librerie commerciali. Out_proj produce half-precision per la normalizzazione successiva. **Non isolate come sensibili specificamente.** **conv1d:** quantizzata a 8-bit su input/weights (memory-bound operator). **RMSNorm:** pesi NON quantizzati (rimangono half-precision); fusion con residual+static quant.

Degrado misurato [MISURATO], baseline = FP16. PPL su WikiText-2: Mamba-2.8B FP16 = 9.45; Quamba W8A8 = 9.91 (+0.46 PPL, +0.28 BPB equivalente approssimato). Pile PPL: FP16 = 6.39 → Quamba = 6.67 (+0.28 PPL). Zero-shot su 6 task (LAMBADA, HellaSwag, PIQA, Arc-E, Arc-C, WinoGrande): Mamba-2.8B FP16 = 63.1% avg → Quamba W8A8 = 62.2% avg (**-0.9%**). Confronto: SmQ-SSM 57.3%, QuaRot-SSM 62.4%, Mamba-PTQ 51.4% (parziale, solo 3 task). Jamba-52B: solo -1% accuracy su LAMBADA. Low-bit: **Quip#-SSM W2A16** su Mamba-2.8B → PPL 12.71 vs FP16 9.45 (**1.34× PPL**); su Llama-2-7b lo stesso metodo dà 1.02× PPL → SSM meno tollerante del Transformer. **QuaRot-SSM W4A4** su Mamba-2.8B → *failed* (modello collassato). Quamba W8A8 = 9.91 (unico funzionante a basso bit-width).

Hardware. Nvidia A5000 GPU (cloud) e Nvidia Orin Nano 8G (edge GPU). Latency 2.8B su Nano: 103.56 ms FP16 → 60.17 ms Quamba W8A8 (**1.72× speedup**). NON testato su CPU x86 consumer. **Cosa copre.** Mamba-1 (130M-2.8B), SSM selettivo, PTQ pesi+attivazioni, quantizzazione a 8-bit/2-bit/4-bit su SSM block, modello large-scale (Jamba-52B), edge GPU profiling. **Cosa NON copre.**

Mamba-2, ternary/W1 pesi, BPB byte-level (solo PPL/accuracy), CPU consumer x86, single-user, SWA, MLP gated-dReLU ternary, pshufb-LUT.

Contraddizioni con le nostre misure. [NEUTRO] sul finding "non isolata in x/dt": Quamba non usa la terminologia *xproj/dtproj* come proiezioni colpevoli; parla di "x" come SSM input activation (tensore post *in_proj*, post *conv1d*, pre discretizzazione). **[CONTRADDIZIONE PARZIALE]** se interpretiamo "x" del cliente come tensore di SSM input: Quamba isola esplicitamente x (input activation) come organo più sensibile. Sul finding 1 (ternary proiezioni = +0.018-0.022 BPB): nessun test di ternary/W1 sui pesi delle proiezioni SSM in Quamba. Il livello più aggressivo testato è W2 (Quip#-SSM) e dà +3.26 PPL su Mamba-2.8B. Non confrontabile direttamente con i nostri +0.018-0.022 BPB (unità diverse: PPL vs BPB; modello diverso; proiezioni separate da tutto il resto). **COPERTURA PARZIALE.**

2.2 Quamba2 — Robust and Scalable PTQ (Mamba-1 + Mamba-2)

Citazione: [11] Chiang, H.-Y.; Chang, C.; Frumkin, N.; Wu, K.-C.; Abdelfattah, M.S.; Marculescu, D. (IST Austria; UT Austin; NYCU), *Quamba2: A Robust and Scalable Post-training Quantization Framework for Selective State Space Models*, ICML 2025 (PMLR v267); arXiv:2503.22879. URL: <https://arxiv.org/abs/2503.22879> — codice: <https://github.com/enyac-group/Quamba>.

Metodo. PTQ (post-training), head-to-toe quantization. Precisioni target: **W8A8, W4A8, W4A16** (+ W4A{8/16}-mixed per mixed-precision). Su **Mamba-1 (1.4B, 2.8B)** e **Mamba-2 (2.7B, 8B)**. ✓ entrambe le generazioni. Tecniche: (i) **sort-and-cluster** per quantizzare SSM input x a 8-bit offline sfruttando channel-persistence + activation persistence; (ii) **per-state-group quantization** per B e C; (iii) **offline Hadamard matrix fusion** su *outproj*; (iv) **head-to-toe** quantizza anche *embedding* e *lmhead* (a differenza di Quamba-1 che li teneva a 16-bit); (v) **W4A{8/16}-mixed** automatico basato su KL sensitivity (auto-search dei blocchi salienti).

Organi quantization-sensitive (CRITICO). SSM input x = ancora identificato come sensibile. Citazione: "It is noted that x continues to be vulnerable to quantization errors in SSMs, consistent with the findings in [chiang2024quamba]". E: "the SSM block is extremely sensitive to quantization-induced errors in xt due to the linear recurrence mechanism in Mamba1. Our work indicates that the phenomenon persists in Mamba2". **B e C** (parametri input-dependent): affrontati con **per-state-group quantization** → 8-bit. Sensibili ma trattabili. **A** (state transition): menzionato come parametro (A,D) trainable ma non esplicitamente sensibile; implicitamente non isolato. **Δ (dt)**: NON menzionato come sensibile; il paper non entra nel merito di Δ-isolation. Appare solo nella formula di discretizzazione. **Projections** (*inproj*, *outproj*, *xproj*, *dt_proj*): trattate come dense layers, quantizzate head-to-toe a 4/8-bit. **Non isolate individualmente. conv1d, RMSNorm:** non discussi individualmente in Quamba2 (erano in Quamba-1).

Degrado misurato [MISURATO], baseline = FP16. Zero-shot 5-task avg (ARC-E, ARC-C, PIQA, WinoGrande, HellaSwag): Mamba-1 1.4B: FP16=58.6% → W8A8 Quamba2=57.5% (-1.1%); W4A8=56.7% (-1.9%); W4A16=57.5% (-1.1%). Mamba-1 2.8B: FP16=62.2% → W8A8=61.8% (-0.4%); W4A8=61.0% (-1.2%); W4A16=61.9% (-0.3%). Mamba-2 2.7B: FP16=62.4% → W8A8=62.1% (-0.3%); W4A8=61.4% (-1.0%); W4A16=62.3% (-0.1%). **Mamba-2 8B:**

FP16=70.8% → W8A8=69.9% (-0.9%); W4A8=69.4% (-1.4%); W4A16=70.2% (-0.6%). MMLU 5-shot Mamba-2 8B: FP16=47.0% → W4A8=41.2% (-5.8%); W4A16=45.3% (-1.7%); W4A{8/16}-mixed Auto=44.0% (-3.0%); LAMBADA 0-shot: FP16=70.9% → W4A8=68.8%. **Headline:** 1.6% accuracy drop avg + 1.3×/3× speedup prefilling/generation + 4× memory reduction.

Hardware. Nvidia A5000 GPU (3× throughput in prefilling su 8B); **Nvidia Orin Nano 8G** (13 TPS su Mamba-2 8B W4A8, 4× memory reduction). NON CPU x86 consumer. **Cosa copre.** Mamba-1, **Mamba-2** (unico fra i paper principali), SSM selettivo, PTQ fino a W4, mixed-precision W4A{8/16}, 8B scale. **Cosa NON copre.** Ternary/W1/W2, BPB byte-level, CPU consumer x86, single-user, SWA, MLP gated-dReLU ternary, pshufb-LUT, analisi esplicita sensitivity per singola proiezione (xproj, dtproj).

Contraddizioni con le nostre misure. [NEUTRO] su finding "non isolata in x/dt": Quamba2 continua a isolare **x** (SSM input) come sensibile. **NON parla di x_proj o dt_proj come colpevoli.** Su dt (Δ) il paper è silente (Δ appare nella discretizzazione ma non è oggetto di analisi di sensitivity). Allineato con il nostro finding se interpretiamo "x" come proiezione x_proj; **[CONTRADDIZIONE PARZIALE]** se interpretiamo "x" come tensore SSM input. Sul finding 1 (ternary proiezioni = +0.018-0.022 BPB): nessun test ternary. Lowest bit-width testato = W4A8 (pesi a 4-bit). Su Mamba-1 2.8B W4A8: -1.2% accuracy zero-shot. Non convertibile in BPB (paper non riporta PPL, solo accuracy). **COPERTURA PARZIALE.**

2.3 KL-Lens — Mixed-Precision Sensitivity (CONTRADDIZIONE PARZIALE)

Citazione: [12] Kong, J.; Pandey, N.P.; Ponzina, F.; Rosing, T. (UC San Diego; SDSU), *A KL Lens on Quantization: Fast, Forward-Only Sensitivity for Mixed-Precision SSM–Transformer Models*, CVPR 2026 Workshop EDGE; arXiv:2604.13440. URL: <https://arxiv.org/abs/2604.13440> — codice: <https://github.com/jasonkongie/kl-ssm-quant>.

Metodo. PTQ + **mixed-precision sensitivity analysis** (non è un paper di recipe di quantizzazione, è un paper di ANALISI di sensitivity). Precisioni: mixed-precision FP16+INT4 (CPU) / FP16+INT8 (GPU); confronto con Uniform INT4 / Uniform INT8 / FP16 baseline. Su **Mamba-130M**, **Mamba-380M**, **Mamba-1.4B**, **Mamba2-130M** (Mamba-1 e Mamba-2) + **Hymba hybrid** (SSM-Transformer-MoE, NVIDIA) + Zamba. Metodo: KL divergence student → teacher come proxy di sensitivity, forward-only (no backprop), assegnazione mista basata su threshold κ .

Organi quantization-sensitive (CRITICO — TABELLA PER SOTTOTIPO). **mamba.x_proj** = **sottotipo PIÙ sensibile**, Δ (avg ablation sensitivity su PPL) = **0.27**, *exceeding all other subtypes by more than an order of magnitude ($\Delta \leq 0.014$)*. **mamba.dt_proj** = **pressoché INSENSIBILE**, Δ = **3.6×10^{-4}** . Citazione: "*suggesting that extremely low-precision representations are feasible*" per dtproj. Altri sottotipi menzionati: *mamba.inproj*, *mamba.outproj*, *MoE up/down* — tutti ≤ 0.014 (analisi su Hymba). ****Localized hotspots****: *block 31 da solo* = ****>70% del sensitivity budget aggregate**** ($\Sigma\Delta = 6.9$); *picchi minori in blocchi 2, 11, 16, 22*. ****Coupled xproj–MoE behaviour****: le sensitività tardive di x_proj co-occorrono con MoE up/down → amplificazione del rumore da routing.

Degrado misurato [MISURATO], baseline = FP16. WikiText-2 PPL su Intel Lunar Lake: Mamba-130M CPU: FP16=21.59 → INT8=25.32 → INT4=3e35 (divergente!) → KL-MP p05=21.61 ($\approx$ FP16). **Mamba-1.4B CPU:** FP16=11.22 → INT8=11.25 → INT4=60.55 → KL-MP p05=**11.22** (perfettamente allineato a FP16). Mamba2-130M GPU: FP16=46.45 → INT8=53.03 → KL-MP p02=46.46. Memory: Mamba-1.4B 5.2 GB (FP16) → 723 MB (INT4) / 1.4 GB (KL-MP) — 7.2× compressione. Latency: Mamba-1.4B 384 ms (FP16) → 196 ms (INT8) → 190 ms (INT4) → **178 ms (KL-MP)** (migliore). Throughput: Mamba-1.4B 2.6 FPS (FP16) → 5.1 FPS (INT8) → 5.3 FPS (INT4) → **5.6 FPS (KL-MP)**. Kendall's τ correlation fra KL divergence e Δ PPL: **0.79** medio (vs SQNR = 0.71), $p < 10^{-6}$.

Hardware. Intel Lunar Lake CPU (consumer x86 NPU/CPU di nuova generazione, Mamba-130M e Mamba-1.4B); GPU (Mamba2-130M). KL-MP p05 batte INT4/INT8 uniform su PPL/latency/throughput. Nota: "*INT8 execution is not supported for the certain relevant operations on Intel Lunar Lake*" per Mamba2-130M GPU.

Cosa copre del nostro caso. Mamba-1, **Mamba-2**, SSM selettivo, **CPU consumer x86** (Intel Lunar Lake), mixed-precision sensitivity, **analisi esplicita per sottotipo di proiezione (x_proj, dt_proj, in_proj, out_proj)** — **unico paper con questa granularità**. **Cosa NON copre.** Ternary/W1 (max INT4), BPB byte-level (usa PPL WikiText-2), single-user esplicito (non dichiarato), SWA, MLP gated-dReLU ternary, pshufb-LUT, Mamba-1 1.4B sì ma non 2.8B su CPU.

[CONTRADDIZIONE FORTE] con il nostro finding "non isolata in x/dt". KL-Lens **isola esplicitamente x_proj come il colpevole primario** ($\Delta=0.27$, $>10\times$ gli altri sottotipi). E **isola anche dt_proj come NON-colpevole** ($\Delta=3.6\times 10^{-4}$). Quindi: contraddice il nostro finding sul lato *xproj* (KL-Lens dice che *xproj* è il colpevole specifico); **allineato** sul lato *dtproj* (entrambi dicono che *dtproj* non è sensibile). Etichetta finale: **[CONTRADDIZIONE PARZIALE]** — contraddice su *xproj*, **allineato** su *dtproj*. Nota: KL-Lens usa Hymba hybrid SSM-Transformer-MoE (architettura diversa dal nostro Mamba-1 puro) e metrica Δ PPL su WikiText-2 (non BPB byte-level) → possibile spiegazione della divergenza.

2.4 Mamba-PTQ — Outlier Channels in Recurrent LLMs

Citazione: [13] Pierro, A.; Abreu, S., *Mamba-PTQ: Outlier Channels in Recurrent Large Language Models*, arXiv:2407.12397, luglio 2024. URL: <https://arxiv.org/abs/2407.12397>. Lavoro preliminare (preprint; citato da Quamba come baseline "Mamba-PTQ" e da KL-Lens come ref [17]).

Metodo. PTQ (post-training), preliminary work. SmoothQuant come outlier-aware baseline. Precisioni: W8 (pesi only) e W8A8 (pesi+attivazioni) su subset di proiezioni; W4 (pesi only) come stress test. Su **Mamba-1** (130M, 1.4B, 2.8B). NON Mamba-2. Metodo: analisi di outlier channel pattern nelle pre-activations delle proiezioni lineari (in, x, out) + valutazione d'impatto della loro rimozione.

Organi quantization-sensitive (CRITICO). Tre proiezioni maggiori (inproj, xproj, outproj): tutte mostrano **outlier channel (<1% dei canali).** Citazione: "The pre-activations of the three largest linear layers (in, x, and out), consistently with what was observed for attention-based LLMs, show outliers accounting for less than 1% of channels." — ****NON isola un singolo colpevole fra le tre****.

****dtproj: _"The linear layer projecting the SSM's time steps (dt) shows almost no outliers"_ — esplicitamente NON sensibile. Outlier channels: se rimossi (zeroing out), drop di accuracy = -12.61% su Mamba-130m e -17.49%**** su Mamba-2.8b → indica che gli outlier (pur essendo <1% dei canali) sono critici per le predizioni del modello. conv1d, RMSNorm, stato h, A/B/C/Δ: NON analizzati (focus solo su proiezioni lineari).

Degrado misurato [MISURATO], baseline = non quantizzato (presunto FP16). One-shot accuracy su 6 task (LAMBADA, HellaSwag, PIQA, WinoGrande, RTE, COPA) su Mamba-1.4B. Baseline LAMBADA = 64.95%. W8 (mlp) = 64.43% (-0.5%); W8A8 (mlp) = 63.11% (-1.8%). **W4 (mlp) = 0.02% (collasso su LAMBADA!)**; W4 (all) = 0.00% (collasso totale su LAMBADA). Solo MLP vs tutto: W4 (mlp) collassa LAMBADA a 0.02% ma PIQA resta 52.29% → indica che W4 distrugge selettivamente LAMBADA. Paper **NON usa PPL** ("we explicitly neglect perplexity benchmarking, since prior work noted how it may not be informative of the actual task performance degradation").

Hardware. NON riportato esplicitamente. Le valutazioni sono su zero-shot accuracy via lm-eval; non c'è profiling hardware. Probabile uso di GPU per inference ma non dichiarato. **Cosa copre.** Mamba-1, SSM selettivo, PTQ baselines su proiezioni (in/x/out/dt), analisi esplicita di dt_proj come NON-outlier (unico oltre a KL-Lens). **Cosa NON copre.** Mamba-2, ternary/W1, BPB, CPU, single-user, SWA, MLP gated-dReLU ternary, pshufb-LUT, analisi di stato h/A/B/C/Δ, profiling hardware.

[ALLINEATO FORTE] con il nostro finding "non isolata in x/dt". Il paper NON isola un singolo colpevole fra le tre proiezioni (in/x/out) e **esplicitamente dice che dt_proj mostra "almost no outliers"**. Allineato perfettamente con il nostro finding "non isolata in x/dt". Sul finding 1 (ternary proiezioni = +0.018-0.022 BPB): W4 (mlp) causa **collasso su LAMBADA** (0.02% da 64.95%) — degrado catastrofico. Su PIQA: 74.16% → 52.29% (-21.9%). Su HellaSwag: 59.11% → 25.72% (-33.4%). Non confrontabile direttamente con i nostri +0.018-0.022 BPB (unità diverse, modello diverso) ma indica che **W4 sulle proiezioni SSM è molto più distruttivo di W8**. Il nostro +0.018-0.022 BPB con ternary (W1⁻) è sorprendentemente piccolo rispetto al collasso W4 osservato qui → suggerisce che il nostro setup (con SWA + MLP ternary + SSM proiezioni fp32 separate) potrebbe avere proprietà di robustezza diverse.

2.5 MambaQuant — Variance Aligned Rotation Methods

Citazione: [14] Xu, T.; Liu, B.; Chen, Z.; Zhang, S.; et al. (gruppo Houmo AI + università cinesi), *MambaQuant: Quantizing the Mamba Family with Variance Aligned Rotation Methods*, ICLR 2025; arXiv:2501.13484. URL: <https://arxiv.org/abs/2501.13484>.

Metodo. PTQ (post-training). Precisioni: **W8A8** e **W4A8** (pesi e attivazioni). No W1/ternary. Su **Mamba-LLM** (370m, 790m, 1.4b, 2.8b) per language; **Vim** e **Mamba-ND** per vision. NON esplicitamente Mamba-2. Metodo: KLT-enhanced rotation (variance-aligned, offline) per gateproj + smooth-fused rotation (online, Hadamard) per outproj e matmul. Risolve il problema che Hadamard puro non allinea le varianze sui canali di Mamba (a differenza dei Transformer).

Organi quantization-sensitive (CRITICO). Gate projection = contiene outlier significativi (Fig.1a). Citazione: "significant outliers which challenge the quantization process are present in gate

projection, output projection, and matrix multiplication". **Output projection (out_proj)** = contiene outlier significativi (Fig.1b, su Vim). **Matrix multiplication (matmul_in1, matmul_in2 — il SSM scan)** = contiene outlier; il **Parallel Scan (PScan)** amplifica gli outlier di un fattore numerico (Fig.13: vim-base outproj passa da 930.3 max original a 1377.8 post-PScan). ****xproj: appare in Fig.13 con valori 15.1 / 12.0 / 11.1 (max activations) — NON identificato come outlier critico; valori modesti rispetto a out_proj (1371.6 / 1064.3 / 930.3).** dtproj**: appare in Fig.13 con valori 14.9 / 13.6 / 4.1 — ****NON identificato come outlier critico****. ****inproj: 49.3 / 48.8 / 38.9 — valori medi.** conv1d**: 48.2 / 36.2 / 36.2 — valori medi. Stato h, A/B/C/Δ: non analizzati esplicitamente per sensitivity.

Degrado misurato [MISURATO], baseline = FP16. Zero-shot avg accuracy su 5 task (ARC-E, ARC-C, PIQA, Winogrande, HellaSwag): Mamba-2.8B: FP16=62.2 → W8A8 (Ours)=62.1 (-0.1%) → W4A8 (Ours)=58.5 (-3.7%). Confronto W8A8: RTN=58.4, GPTQ+RTN=58.9, SmoothQuant=58.7, QuaRot=59.3. Mamba-1.4B: FP16=58.6 → W8A8=58.3 (-0.3%) → W4A8=54.3 (-4.3%). Mamba-790M: FP16=54.8 → W8A8=53.8 (-1.0%) → W4A8=45.8 (-9.0%). Mamba-370M: FP16=50.9 → W8A8=50.0 (-0.9%) → W4A8=43.9 (-7.0%). Headline: "less than 1% accuracy loss" su W8A8 per vision+language.

Hardware. GPU (non specificata nel dettaglio estratto). Focus su accuracy, non su latency profiling. Mamba-2.8B preso come reference per overhead computazionale: +0.01% parametri, +0.91% FLOPs. **Cosa copre.** Mamba-1, SSM selettivo, PTQ W8A8/W4A8, analisi di outlier per singola proiezione (gate, out, x, dt, in, conv1d), Mamba-LLM fino a 2.8B. **Cosa NON copre.** Mamba-2 (solo Mamba-1 e Vim/Mamba-ND vision), ternary/W1, BPB, CPU consumer, single-user, SWA, MLP gated-dReLU ternary, pshufb-LUT.

[ALLINEATO PARZIALE] con il nostro finding "non isolata in x/dt". MambaQuant **NON isola x_proj o dt_proj come colpevoli**; identifica invece **gate_proj + out_proj + matmul** come i contenitori di outlier. xproj e dtproj hanno valori modesti in Fig.13. Quindi allineato con il nostro finding "non isolata in x/dt". Sul finding 1 (ternary proiezioni = +0.018-0.022 BPB): nessun test ternary. Lowest = W4A8 → -3.7% accuracy su Mamba-2.8B. Non confrontabile con BPB.

2.6 Q-Mamba — PTQ with State Cache Quantization (secondario)

Citazione: [15] Chen, T.; Chen, Y.; Wang, P.; Xu, W.; Zhu, Z.; Cheng, J., *Q-Mamba: Towards more efficient Mamba models via post-training quantization*, Findings of ACL 2025, pp. 10594-10610. DOI: 10.18653/v1/2025.findings-acl.551. URL: <https://aclanthology.org/2025.findings-acl.551/>. (PDF non estraibile in questa sessione via web-reader; dati presi dall'abstract ACL Anthology + OpenReview snippet).

Metodo. PTQ (post-training). Precisione: **W8A8H4** (8-bit pesi, 8-bit attivazioni, **4-bit state caches** H) — unico paper che quantizza anche gli state caches. Su **Mamba-2 2.7B** (e altri sizes non specificati nell'abstract). Tecniche: **Decoupled Scale Quantization (DSQ)** per outlier nelle dimensioni state+channel; **Efficient Selectivity Reconstruction (ESR)** per preservare selective ability con parallel scan + non-linear quant function.

Organi quantization-sensitive. Linear projections: quantizzate (generic; non isola xproj/dtproj). **State caches (h):** identificati come fonte significativa di memory consumption dopo aver quantizzato

le proiezioni. Outlier nelle dimensioni state+channel → affrontati con DSQ. **Analisi teorica delle cause di outlier negli stati** (PDF non estraibile per dettagli). Stato h/A/B/C/Δ singolarmente: non dettagliati nell'abstract. **Degrado misurato [MISURATO]**. Mamba-2 2.7B W8A8H4: **50% riduzione memory + 2.13% accuracy degradation** su zero-shot tasks. Dettagli su altri size/config non estraibili (PDF 0 byte via reader). **Hardware**: non dichiarato nell'abstract.

Cosa copre. Mamba-2, SSM selettivo, **quantizzazione state cache** (unico paper), PTQ W8A8H4. **Cosa NON copre**. Ternary/W1, BPB, CPU consumer, single-user, SWA, MLP gated-dReLU ternary, pshufb-LUT, analisi esplicita per sottotipo proiezione. **[NEUTRO]** — l'abstract non entra nel merito di sensitivity per singola proiezione.

2.7 Sintesi — Convergenze e divergenze

Convergenza forte: SSM input x (tensore, non proiezione) è sensibile agli errori di quantizzazione nella linear recurrence — Quamba, Quamba2, KL-Lens tutti concordano. **Divergenza sul finding "non isolata in x/dt"**:

- **[CONTRADDIZIONE PARZIALE]**: KL-Lens isola **x_proj** come colpevole primario ($\Delta=0.27$, $>10\times$ gli altri sottotipi) — questo contraddice il nostro finding sul lato *xproj*. *Ma KL-Lens **conferma** il nostro finding sul lato dtproj* ($\Delta=3.6\times 10^{-4}$ = quasi insensibile).
- **[ALLINEATO FORTE]**: Mamba-PTQ esplicitamente dice *dtproj ha "almost no outliers"* e non isola singolo colpevole fra in/x/out.
- **[ALLINEATO PARZIALE]**: MambaQuant non isola *xproj o dtproj*; concentra su *gateproj + outproj + matmul*.
- **[NEUTRO]**: Quamba/Quamba2 parlano di "x" come tensore SSM input, non come proiezione *x_proj*; etichetta ambigua.

Degrado misurato per il nostro caso (proiezioni ternarizzate): NESSUN paper testa W1/ternary pesi sulle proiezioni SSM. Più vicini: Quamba Tab.7: Quip#-SSM W2A16 su Mamba-2.8B → PPL 12.71 vs 9.45 FP16 ($1.34\times$ PPL). QuaRot-SSM W4A4 → *failed* (collasso). Mamba-PTQ W4 (mlp) su Mamba-1.4B → LAMBADA collassa a 0.02% da 64.95%. Quamba W8A8 → +0.46 PPL su WikiText-2 (+0.28 BPB equivalente approssimato su Mamba-2.8B). I nostri +0.018-0.022 BPB con ternary sono **molto più piccoli** dei degradi W4/W2 riportati in letteratura → il nostro setup (SWA + MLP ternary + SSM proiezioni fp32 separate) potrebbe avere proprietà di robustezza diverse; oppure il nostro esperimento ha una potenza statistica che non riesce a vedere il collasso totale. Da dichiarare come gap.

Hardware: solo **KL-Lens** usa Intel Lunar Lake consumer CPU. Quamba/Quamba2 su Orin Nano edge GPU. MambaQuant/Mamba-PTQ su GPU non specificata. **NESSUN paper** usa CPU consumer x86 single-user con pshufb-LUT. **Zona bianca specifica del cliente**: misurazione diretta del degrado di ternarizzazione (W1⁻) delle proiezioni SSM in regime BPB byte-level su modello Mamba-1 con SWA + MLP gated-dReLU ternary. La letteratura conferma che le proiezioni SSM sono sensibili (KL-Lens: *x_proj* dominate; Quamba/Quamba2: SSM input x sensibile) ma nessuno le ha ternarizzate con misurazione BPB. Il nostro +0.018-0.022 BPB è un dato non coperto direttamente.

3. Domanda 3 — Training ternario QAT a piccola scala

Otto paper primari + 2 secondari identificati. Coprono: BitNet originale (binario, 125M-30B), BitNet b1.58 (ternary, 3B-70B), BitNet v2 (ternary+4-bit act, 400M-7B), Spectra/TriLM (ternary, 99M-3.9B, più vicino al nostro anchor), TernaryBERT (ternary+distillazione, 14.5M-109M), TWN (ternary CNN, ~50K-16M), BitDelta (binary delta, 7B+), LUT-NN (centroid, non ternary puro), PT²-LLM (PTQ ternary, 7B+, controllo negativo), BinaryBERT (binary BERT, menzione). **Trend convergente**: il costo di qualità del ternary CALA al crescere della scala (QAT); il contrario per PTQ.

3.1 BitNet — Scaling 1-bit Transformers (framing, binary)

Citazione: [3] Wang, H.; Ma, S.; Dong, L.; Huang, S.; Wang, H.; Ma, L.; Yang, F.; Wang, R.; Wu, Y.; Wei, F. (Microsoft Research + UWashington + USTC + CAS), *BitNet: Scaling 1-bit Transformers for Large Language Models*, arXiv:2310.11453; JMLR vol. 26 (24-2050), 2025. URL: <https://arxiv.org/abs/2310.11453>.

Metodo. QAT — training **da zero** ("*BitLinear as drop-in replacement of nn.Linear to train 1-bit weights from scratch*"). **Binary** {-1, +1} weights, 8-bit activations (W1A8). Non ternary. Transformer (autoregressive LM, sostituisce nn.Linear con BitLinear). Pesi + attivazioni quantizzati (pesi 1-bit, attivazioni INT8).

Scala testata. **125M, 355M (implicito), 1.3B, 6.7B, 13B, 30B** — serie da 125M a 30B. Evidenza su <100M: **NO** (125M è il più piccolo, appena sopra). Evidenza su 5M: **NO**. Linguaggio: language modeling su Pile + Common Crawl + RealNews + CC-Stories; vocab 16K SentencePiece.

Costo di qualità misurato [MISURATO], baseline = FP16 Transformer. Zero-shot accuracy su Winogrande/StoryCloze/HellaSwag/PIQA a 6.7B vs FP16 Transformer + vs PTQ (Absmax, SmoothQuant, GPTQ, QuIP). Metrica riportata: PPL su Pile validation + zero-shot accuracy a 6.7B. BitNet W1A8 supera W1A8 PTQ baselines (Absmax/SmoothQuant W1A8) e compete con FP16. **BPB**: non riportato. Trend esplicito sul loss scaling law: power-law $L(N)=aN^b+c$ come FP16.

Trend di scala (CRITICO). COSTO CALA al crescere della scala. Citazione diretta: "*the gap between BitNet and FP16 Transformer becomes smaller as the model size grows*" (Sez. 3.2). La scaling law segue la stessa power-law di FP16 con termine irreducibile c, e il differenziale si riduce con N. **Giustificazione dichiarata**: sì, "*as the model size scales up, the cost savings become more significant while achieving competitive performance*".

Confronto col nostro anchor. Anchor: +0.013 BPB a 5M (vs fp32). BitNet riporta PPL su Pile + zero-shot accuracy, **non BPB**. Conversione approssimata: 1 BPB $\approx \times 1.44$ PPL factor $\rightarrow$ +0.013 BPB $\approx \times 1.005$ PPL ($\approx +0.5\%$ PPL). BitNet non scende sotto 125M, quindi non può validare direttamente il nostro numero a 5M. Tuttavia la direzione del trend (gap cala con scala) è **CONSISTENTE** con un gap piccolo ma non-zero a 5M. **Allineato, non contraddittorio**. Hardware: training su GPU; inference energy stimata a 7nm node. **Cosa copre**: QAT da zero, Transformer LM, small scale <100M PARZIALE (125M). **Cosa NON copre**: ternary {-1,0,+1} (è binario), scala 5M, BPB byte-level, gated-dReLU, SSM, single-user CPU.

3.2 BitNet b1.58 — The Era of 1-bit LLMs (ternary, fondativo)

Citazione: [16] Ma, S.; Wang, H.; Ma, L.; Wang, L.; Wang, W.; Huang, S.; Dong, L.; Wang, R.; Xue, J.; Wei, F. (Microsoft Research + UChinese Academy of Sciences), *The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits*, arXiv:2402.17764, 27 febbraio 2024. URL: <https://arxiv.org/abs/2402.17764>.

Metodo. QAT — training **da zero**. Ternary $\{-1, 0, +1\}$ weights con absmean quantization + RoundClip; 8-bit activations (W1.58A8). Transformer LLaMA-style (BitLinear al posto di nn.Linear; RMSNorm, SwiGLU, RoPE). Pesi + attivazioni quantizzati (embedding rimane full-precision).

Scala testata. 3B, 7B, 13B, 70B (confronto con LLaMA 1B/7B/13B/70B riprodotto). Evidenza su <100M: **NO** (3B è il più piccolo). Evidenza su 5M: **NO**. Linguaggio: RedPajama 100B tokens (pretraining), più esperimento StableLM-3B recipe a 2T tokens.

Costo di qualità misurato [MISURATO], baseline = FP16 LLaMA riprodotto. Citazione: *"BitNet b1.58 can match full precision (i.e., FP16) baselines in terms of both perplexity and end-task performance, starting from a 3B size"*. Metrica: WikiText2 + C4 validation **perplexity**, zero-shot accuracy su ARC-Easy/ARC-Challenge/HellaSwag/Winogrande/PIQA/OpenbookQA/BoolQ. **BPB:** non riportato.

Trend di scala (CRITICO). COSTO CALA al crescere della scala. Claim esplicito: il match con FP16 inizia **"a partire da 3B"** → implicitamente sotto 3B c'è un gap misurabile (non quantificato nel paper). **Giustificazione dichiarata:** *"modeling capability is stronger due to its explicit support for feature filtering, made possible by the inclusion of 0 in the model weights, which can significantly improve the performance of 1-bit LLMs"*. Trend Pareto esplicito su latency/memory/throughput/energy: lo speedup **cresce** con scala (BitNet b1.58 70B è 4.1× più veloce di LLaMA 70B).

Confronto col nostro anchor. Anchor +0.013 BPB a 5M. BitNet b1.58 non scende sotto 3B → **non può validare o confutare direttamente** il nostro numero a 5M. Per estrapolazione del trend (gap cala con scala): a 5M ci si aspetta un gap > gap a 3B (che è ≈0), quindi >0. L'anchor (+0.013 BPB ≈ +0.5% PPL) è piccolo ma >0 → **consistente con il trend. Allineato, non contraddittorio.** Hardware: training su GPU (cluster Microsoft); inference NVIDIA FasterTransformer + Ladder 2-bit kernel. **Cosa copre:** ternary $\{-1, 0, +1\}$, QAT da zero, Transformer LM, LLaMA-style (SwiGLU gated MLP) — simile al nostro gated-dReLU (ma non identico). **Cosa NON copre:** scala 5M (min 3B), scala <100M, BPB byte-level, dReLU esplicito, SSM, single-user CPU.

3.3 BitNet v2 — Native 4-bit Activations (ternary + 4-bit act, scala minore)

Citazione: [17] Wang, H.; Ma, S.; Wei, F. (Microsoft Research + UChinese Academy of Sciences), *BitNet v2: Native 4-bit Activations with Hadamard Transformation for 1-bit LLMs*, arXiv:2504.18415, 13 giugno 2025. URL: <https://arxiv.org/abs/2504.18415>.

Metodo. QAT — training **da zero**. Ternary $\{-1,0,+1\}$ weights (eredita da b1.58); attivazioni native 4-bit tramite Hadamard transformation (H-BitLinear). Transformer LLaMA-style. Pesi + attivazioni quantizzati.

Scala testata. 400M, 1.3B, 3B, 7B (esplicitamente tabulati in Table 1). Evidenza su <100M: **NO** (400M è il più piccolo, ma è il più piccolo tra i paper BitNet-family). Evidenza su 5M: **NO**. Linguaggio: RedPajama 100B tokens.

Costo di qualità misurato [MISURATO], baseline = BitNet b1.58 (riferimento ternary). Tabella 1 riporta PPL su C4 validation + zero-shot su ARCC/ARCE/HellaSwag/PIQA/Winogrande/LAMBADA. Numeri chiave (PPL C4): 400M: BitNet b1.58 = 13.37; BitNet v2 (a8) = 13.50; BitNet v2 (a4) = 13.78. 1.3B: b1.58 = 11.02; v2 (a8) = 11.14; v2 (a4) = 11.33. 3B: b1.58 = 9.71; v2 (a8) = 9.72; v2 (a4) = 9.85. 7B: b1.58 = 9.09; v2 (a8) = 9.14; v2 (a4) = 9.24. Match con FP16 non è il focus di v2. **BPB:** non riportato.

Trend di scala (CRITICO). Il costo del 4-bit activation **CALA al crescere della scala:** gap PPL v2(a4) vs v2(a8) a 400M = +0.28; a 7B = +0.10. Trend confermato anche per b1.58 vs v2(a8) (gap quasi zero a 3B+). **Giustificazione dichiarata:** sì, *"minimal performance degradation when trained with native 4-bit activations, significantly reducing memory footprint and computational cost for batched inference"*.

Confronto col nostro anchor. Il paper testa a 400M (più vicino a 5M dei precedenti, ma ancora 80× più grande). A 400M il gap tra b1.58 (riferimento ternary) e v2 (a4) è +0.28 PPL C4. Conversione approssimata +0.28 PPL $\approx$ +0.06 BPB (assumendo ~ 4 bytes/token). Più alto del nostro +0.013 BPB a 5M — ma le metriche non sono direttamente paragonabili (v2 confronta a4 vs a8, non ternary vs fp32). Anchor vs fp32: +0.013 BPB $\rightarrow$ consistente col fatto che il costo *diminuisce* con scala, ma richiede estrapolazione. **Allineato, non contraddittorio** (ma con riserva: la metrica di v2 è costo del 4-bit activation, non costo del ternary vs fp32).

3.4 Spectra/TriLM — Pretraining Ternary LM at Scale (più vicino al nostro anchor)

Citazione: [18] Kaushal, A.; Vaidya, T.; Mondal, A.K.; Pandey, T.; Bhagat, A.; Rish, I. (Nolano Research / MILA), *Spectra: Surprising Effectiveness of Pretraining Ternary Language Models at Scale*, arXiv:2407.12327, ICLR 2025 Spotlight. URL: <https://arxiv.org/abs/2407.12327>.

Metodo. QAT — pretraining **da zero**. TriLM = LLaMA-style con **ternary $\{-1,0,+1\}$ weights** + un singolo scale FP16 per weight matrix. Ternary $\{-1,0,+1\}$ weights; attivazioni **non quantizzate** (rimangono FP16). Quantizzazione pesi solo. Transformer LLaMA-style (RMSNorm, **SwiGLU gated MLP**, RoPE, MHA, no bias) — l'architettura gated-MLP è molto affine al nostro gated-dReLU. Metodo QAT: latent master weights in FP, absmean scale, ternarize on-the-fly in forward, Straight-Through Estimator in backward.

Scala testata. 99M, 270M, 350M, 760M, 830M, 1.1B, 1.5B, 2.4B, 3.9B — 9 dimensioni. **99M è la scala più piccola nella letteratura ternary-LM.** Evidenza su <100M: **PARZIALE** (99M, appena sopra soglia). Evidenza su 5M: **NO**. Linguaggio: 300B SlimPajama tokens, identical training data/ordering per tutti i modelli.

Costo di qualità misurato [MISURATO], baseline = FloatLM (FP16) stessa parameter count e training data. Confronto TriLM vs FloatLM (FP16) vs QuantLM (3,4,6,8-bit PTQ) alla stessa parameter count. Metriche: validation loss/perplexity (PTB, LAMBADA, PennTreeBank, SlimPajama validation), zero-shot accuracy su 8 commonsense/reasoning benchmarks (ARC-Easy, ARC-Challenge, BoolQ, HellaSwag, Winogrande, PIQA, LAMBADA, LogiQA), MMLU, SciQ, TriviaQA. **BPB:** non riportato (usa validation loss/PPL). Claim chiave: *"TriLM 3.9B matches the performance of FloatLM 3.9B across all benchmarks, despite having fewer bits than FloatLM 830M"*.

Trend di scala (CRITICO). COSTO CALA al crescere della scala. Trend esplicito: **Sotto 1B** parametri: TriLM ha un gap misurabile in validation loss vs FloatLM. Citazione: *"the difference between the two [FloatLM vs TriLM] considerably narrows at Billion+ parameter scale and the trends shows the potential for TriLMs to meet (or even outperform) FloatLMs of same parameter count"*. **Sopra 2.4B:** TriLM batte FloatLM su LAMBADA e commonsense avg, ma resta indietro su memorizzazione (SlimPajama validation, MMLU). **3.9B:** match su commonsense & knowledge, gap su memorizzazione/in-distribution. **Giustificazione dichiarata:** sì, *"TriLMs offer superior scaling behavior in terms of model size (in bits). Surprisingly, at scales exceeding one billion parameters, TriLMs consistently outperform their QuantLM and FloatLM counterparts for a given bit size"*.

Confronto col nostro anchor. Anchor $+0.013 \pm 0.005$ BPB a 5M vs FP32. TriLM non scende sotto 99M e non usa BPB → **validazione diretta impossibile**. Per **estrapolazione del trend** (gap misurabile sotto 1B, gap → 0 sopra 1B): a 5M ci si aspetta un gap > gap a 99M. Il nostro $+0.013$ BPB a 5M è un numero piccolo ma non-zero, **consistente** con un gap che cresce al decrescere della scala. Direzione del trend: ALLINEATA. Magnitudo: non verificabile perché il paper non scende sotto 99M e usa PPL non BPB. **Allineato, non contraddittorio.** Hardware: training su GPU (cluster non specificato). 54 modelli × 300B tokens. **Cosa copre:** ternary $\{-1,0,+1\}$, QAT da zero, Transformer LM, **gated MLP (SwiGLU)** — affine al nostro gated-dReLU, **small scale <100M PARZIALE (99M, migliore della letteratura)**. **Cosa NON copre:** scala 5M (min 99M, ~20× più grande), BPB byte-level, dReLU esplicito (SwiGLU, non dReLU), SSM, single-user CPU.

3.5 TernaryBERT — Distillation-aware Ultra-low Bit BERT

Citazione: [19] Zhang, W.; Hou, L.; Yin, Y.; Shang, L.; Chen, X.; Jiang, X.; Liu, Q. (Huawei Noah's Ark Lab), *TernaryBERT: Distillation-aware Ultra-low Bit BERT*, EMNLP 2020 (Findings); arXiv:2009.12812. URL: <https://arxiv.org/abs/2009.12812>.

Metodo. PTQ + distillazione: ternarizza pesi di un BERT **fine-tunato** (non pretraining da zero). Usa sia approximation-based (TWN) che loss-aware (LAT) ternarization + knowledge distillation da TinyBERT. **Ternary $\{-1,0,+1\}$** weights, 2-bit weights + 2-bit o 8-bit activations (config W2A2 o W2A8). Transformer BERT (encoder, non autoregressive). Pesi (Transformer + embedding) + attivazioni quantizzati.

Scala testata. BERT-base 109M (TernaryBERT); BERT-large menzionato ma non primario. TernaryTinyBERT 14.5M (TinyBERT distillato poi ternarizzato) è il più piccolo testato. Evidenza su

<100M: **PARZIALE** (TernaryTinyBERT 18 MB $\approx$ 14.5M parametri). Evidenza su 5M: **NO**. NLP **classification** (GLUE) + QA (SQuAD), **non language modeling**.

Costo di qualità misurato [MISURATO], baseline = FP32 BERT-base. TernaryBERT GLUE avg (BERT-base 109M): FP32 = 78.2 $\rightarrow$ Ternary W2A32 = 77.3; Ternary W2A8 = 76.9. Costo: **-0.9 to -1.3 punti accuracy avg** ($\approx$ -1.2% a -1.7% relativo). SQuAD: gap F1 minore ($\approx$ -1 punto). **BPB:** non riportato (usa accuracy).

Trend di scala. Trend di scala **non esplicito**: testato a una sola scala principale (BERT-base 109M). TernaryTinyBERT 14.5M è menzionato ma non c'è analisi sistematica "*gap vs scala*". **Giustificazione dichiarata:** non riportata esplicitamente per scaling; focus è su usare distillation per ridurre il degradation. **Confronto col nostro anchor.** Anchor +0.013 BPB a 5M vs FP32. TernaryBERT riporta accuracy (non BPB), quindi conversione impossibile. Su BERT-base 109M il costo è $\sim$ 1% accuracy drop. Su TinyBERT 14.5M il costo è comparabile o leggermente maggiore. **Trend qualitativo:** gap $\sim$ 1% accuracy a $\sim$ 14-109M è **lo stesso ordine di grandezza** del nostro +0.013 BPB ($\approx$ +0.5% PPL) a 5M. **Allineato come ordine di grandezza**, ma metriche non confrontabili direttamente. Hardware: training su GPU (Huawei Ascend AI + MindSpore). **Cosa copre:** ternary $\{-1, 0, +1\}$, small scale <100M PARZIALE (14.5M TernaryTinyBERT). **Cosa NON copre:** QAT da zero su LM (è PTQ + distillazione su BERT classification, non pretraining LM), scala 5M, BPB, gated-dReLU, SSM, single-user CPU. **[NOTA]:** TernaryBERT usa distillazione da un modello FP32 grande $\rightarrow$ il costo misurato è dopo distillazione, quindi sottostima il costo di un QAT puro da zero. Il nostro anchor è QAT puro $\rightarrow$ non direttamente confrontabile in magnitudo.

3.6 TWN — Ternary Weight Networks (CNN, fondativo)

Citazione: [20] Li, F.; Liu, B.; Wang, X.; Zhang, B.; Yan, J. (Tsinghua / SJTU / NUDT), *Ternary Weight Networks*, arXiv:1605.04711. URL: <https://arxiv.org/abs/1605.04711>.

Metodo. QAT — training **da zero** con threshold-based ternary function ottimizzata per minimizzare distanza euclidea tra pesi FP e ternari + scaling factor. **Ternary $\{-1, 0, +1\}$ weights + scaling factor; attivazioni non quantizzate** (FP32 activations). **CNN** (LeNet-5, VGG-7, ResNet18, ResNet18B) — non Transformer. Quantizzazione **pesi solo**.

Scala testata. LeNet-5 ($\sim$ 50K params, MNIST), VGG-7 ($\sim$ 10M params, CIFAR-10), ResNet18 ($\sim$ 11M, ImageNet), ResNet18B 1.5 $\times$ ($\sim$ 16M, ImageNet). Evidenza su <100M: **SÌ** — tutti i modelli testati sono <20M. Evidenza su 5M: **NO** esplicito, ma l'ordine di grandezza è simile (LeNet-5 è molto più piccolo di 5M, ResNet18 $\sim$ 11M è vicino). **Vision classification** (MNIST, CIFAR-10, ImageNet, PASCAL VOC detection), non language.

Costo di qualità misurato [MISURATO], baseline = FP32 FPWN (Full Precision Weight Network). Risultati chiave (top-1 accuracy): MNIST (LeNet-5): FPWN 99.41 $\rightarrow$ TWN 99.35 (costo -0.06%). CIFAR-10 (VGG-7): FPWN 92.88 $\rightarrow$ TWN 92.56 (costo -0.32%). ImageNet (ResNet18): FPWN 65.4 $\rightarrow$ TWN 61.80 (costo -3.6%). ImageNet (ResNet18B 1.5 $\times$): FPWN 67.6 $\rightarrow$ TWN 65.3 (costo -2.3%). **BPB:** non riportato (usa accuracy).

Trend di scala. Trend osservabile tra ResNet18 (11M) e ResNet18B (16M): costo -3.6% $\rightarrow$ -2.3%. **COSTO CALA al crescere della scala** (da 11M a 16M). Trend simile tra CIFAR-10 (VGG-7 $\sim$ 10M,

gap -0.32%) e ImageNet (ResNet18 ~11M, gap -3.6%) — ma task diversi, confronto non pulito. **Giustificazione dichiarata:** il paper motiva la maggiore capacità espressiva del ternary vs binary ($3^9=19683$ templates per 3×3 filter vs $2^9=512$), ma non fa analisi esplicita di scaling law. **Confronto col nostro anchor.** Anchor +0.013 BPB a 5M (language) vs FP32. TWN testa a 11-16M (vision) con gap -2.3% to -3.6% accuracy. Settore diverso (vision classification vs language modeling) e metrica diversa (accuracy vs BPB) → confronto diretto impossibile. Ordine di grandezza: gap ~2-4% accuracy su ResNet18-18B è maggiore del nostro +0.5% PPL a 5M. Ma il task è più semplice (classification) e il modello è molto più piccolo di un LM comparabile. **Non confrontabile direttamente;** non c'è contraddizione formale. Hardware: training su GPU (4 GPU per ImageNet). **Cosa copre:** ternary $\{-1,0,+1\}$, QAT da zero, small scale <100M Sì (tutti i modelli <20M). **Cosa NON copre:** language modeling (è vision), Transformer (CNN), scala 5M, BPB, gated-dReLU, SSM, single-user CPU.

3.7 PT²-LLM — Post-Training Ternarization (controllo negativo)

Citazione: [21] Yan, X.; Bao, C.; Li, Z.; Zhang, T.; Yang, K.; Qin, H.; Xie, R.; Sun, X.; Zhang, Y. (SJTU + ETH Zürich + Tencent Hunyuan), *PT²-LLM: Post-Training Ternarization for Large Language Models*, arXiv:2510.03267, 27 settembre 2025. URL: <https://arxiv.org/abs/2510.03267>.

Metodo. PTQ (post-training, NON QAT) con Asymmetric Ternary Quantizer + Iterative Ternary Fitting + Activation-aware Grid Alignment + Structural Similarity-based Reordering. Ternary weights, ~1.58 bit. LLaMA / LLaMA-2 / LLaMA-3 / Qwen3. Solo pesi quantizzati (attivazioni non quantizzate in questo lavoro).

Scala testata. LLaMA-7B, 13B, 70B. Evidenza su <100M: **NO**. Evidenza su 5M: **NO**. **Costo di qualità misurato [MISURATO], baseline = FP16.** L-7B: FP16 Wiki2 PPL 5.68 → PT²-LLM (1.58-bit) Wiki2 PPL 11.39 (~2× peggio); zero-shot avg 61.73 → 45.07 (-16.7 pts). L-13B: FP16 PPL 5.09 → PT²-LLM PPL 11.39. **BPB:** non riportato.

Trend di scala. Costo resta alto su tutte le scale 7B-70B. **Non c'è calo del costo con la scala** — anzi, su 70B la PPL peggiora in modo simile. Questo è il contrario del trend QAT. **Confronto col nostro anchor. [CONTRADDIZIONE TENDENZIALE]:** PTQ ternary mostra gap enorme (~2× PPL) che NON si chiude con scala, mentre QAT ternary (BitNet b1.58, Spectra) mostra gap che si chiude sopra 1-3B. La differenza è QAT vs PTQ. Il nostro anchor è QAT → allineato con trend QAT, in contraddizione con trend PTQ. Citazione diretta dal paper: "*While attempts have been made to extend QAT-based ternarization to LLMs (e.g., BitNet b1.58), such approaches are highly impractical due to the immense parameter scale and the prohibitive demands on training resources*" — PT²-LLM motivates PTQ perché QAT è impractical. Hardware: GPU (LLaMA family). **Cosa copre:** niente di rilevante per QAT ternary small scale (è PTQ su 7B+). **Cosa NON copre:** QAT, small scale, pretraining, BPB. **[CONTRADDIZIONE TENDENZIALE]:** il paper mostra che PTQ ternary NON segue il trend "*gap cala con scala*" — rimane alto. Questo **conferma indirettamente** che il trend positivo (gap cala con scala) è specifico del QAT e non della ternarizzazione in generale.

3.8 Altri (BitDelta, LUT-NN, BinaryBERT — menzioni secondarie)

BitDelta [22] Liu, J. et al. (MIT + Princeton + NVIDIA + Together AI), *BitDelta: Your Fine-Tune May Only Be Worth One Bit*, arXiv:2402.10193, 2024. URL: <https://arxiv.org/abs/2402.10193>. **PTQ post-fine-tuning**: decompone $W_{\text{finetuned}} = W_{\text{base}} + \Delta$, quantizza **solo il delta Δ** a 1-bit (sign quantization) + scale distillation. **Binary $\{-1,+1\}$** sul delta (non ternary). Llama-2-7B/13B/70B, Mistral-7B, MPT-7B. L-7B TruthfulQA: 50.38 $\rightarrow$ 48.75 (-1.6 pts); GSM8K: 14.18 $\rightarrow$ 14.48 (pari). **Non confrontabile**: BitDelta comprime il delta di fine-tuning, non i pesi del modello base. Il nostro anchor è su pretraining QAT ternary dei pesi del modello. Tecnologia e scenario diversi.

LUT-NN [4] (già citato in Domanda 1). QAT con **centroid learning** + table lookup inference. Effective bit-width variabile (1/2/4-bit via codebook size). ResNet18, SENET18, BERT. Accuracy gap: da +1.98% (Speech Commands) a **-2.42% (ImageNet)**. **Non confrontabile** direttamente con il nostro anchor: LUT-NN non è ternary puro $\{-1,0,+1\}$ — usa codebook con quantizzazione centroid-based. Diverso paradigma.

BinaryBERT [23] Bai, H. et al. (Huawei Noah's Ark Lab + CUHK), *BinaryBERT: Pushing the Limit of BERT Quantization*, ACL 2021; arXiv:2012.15701. URL: <https://arxiv.org/abs/2012.15701>. **Binary $\{-1,+1\}$** weights su BERT-base 109M (417 MB $\rightarrow$ 13.4 MB, 24× compressione). Metodo: **ternary weight splitting** — inizializza BinaryBERT da un modello ternary half-sized + split + fine-tuning. Glue avg: FP32 83.9 $\rightarrow$ Binary W1A8 (TWS) 82.7; W1A4 82.2. Costo ~ -1.2 to -1.7 pts avg GLUE. Baseline FP32 BERT-base. Distillation coinvolto. Non è QAT puro da zero su LM (è BERT classification con splitting trick). Small scale: 109M, no <100M, no 5M, no BPB, no dReLU. **Allineato come ordine di grandezza** al nostro anchor (gap $\sim 1\%$ su $\sim 100\text{M}$), ma metriche e task non confrontabili.

3.9 Sintesi — Trend scala QAT ternary

Trend di scala — conferma convergente. Tutti i paper **QAT ternary** mostrano che il **costo di qualità CALA al crescere della scala**:

- **BitNet b1.58**: match FP16 a 3B+ (gap $\rightarrow$ 0 sopra 3B).
- **BitNet (originale, binary)**: "*gap becomes smaller as the model size grows*", testato 125M-30B.
- **Spectra/TriLM**: gap misurabile sotto 1B, gap $\rightarrow$ 0 sopra 1B per commonsense/knowledge (resta gap su memorizzazione a 3.9B).
- **BitNet v2**: gap tra 4-bit e 8-bit activation cala da 400M a 7B.
- **TWN (CNN)**: gap ResNet18 $\rightarrow$ ResNet18B cala da -3.6% a -2.3%.

Trend opposto in PTQ. PT²-LLM (PTQ ternary): gap resta $\sim 2\times$ PPL su 7B-70B, **non si chiude** con scala. Conferma che il trend "*gap cala con scala*" è specifico del QAT.

Confronto con anchor cliente. Anchor: $+0.013 \pm 0.005$ BPB combinato ternary+dReLU a 5M vs fp32. **Coerenza con trend**: ALLINEATO. Sotto 1B (e quindi a maggior ragione a 5M), la letteratura prevede un gap piccolo ma non-zero. $+0.013$ BPB $\approx \times 1.005$ PPL ($\approx +0.5\%$ PPL) è **piccolo ma non-zero** $\rightarrow$ consistente con estrapolazione del trend Spectra/TriLM al di sotto di 99M. **Validazione diretta**: IMPOSSIBILE. Nessun paper ternary QAT scende sotto 99M (Spectra/TriLM è il minimo).

Nessun paper riporta BPB byte-level. Scala 5M è territorio non coperto. **Contraddizioni:** NESSUNA formale. Solo **[NOTA DI CAUTELA]**: se la curva di scaling fosse molto ripida, a 5M ci si aspetterebbe gap molto maggiore di +0.013 BPB — ma la letteratura non quantifica il punto sotto 99M.

Gap di evidenza (da dichiarare nel dossier). (1) Scala 5M: NESSUN paper ternary QAT testa a 5M. Spectra/TriLM (99M) è il più vicino (20× più grande). (2) BPB byte-level: NESSUN paper riporta BPB — tutti usano PPL su WikiText2/C4 o zero-shot accuracy. Conversione richiede assunzioni sul tokenizer. (3) dReLU gated MLP: NESSUN paper usa dReLU esplicito. Spectra/TriLM (SwiGLU) e BitNet b1.58 (SwiGLU) usano gated MLP simile, ma non dReLU. BitNet originale usa standard MLP. (4) SSM: NESSUN paper QAT ternary è su SSM (tutti Transformer). (5) Single-user CPU: NESSUN paper QAT ternary testa single-user CPU. BitNet.cpp (ACL 2025, paper separato) testa CPU inference ma non training.

4. Tabella riassuntiva lavoro × dimensioni

La tabella sintetizza i 23 lavori estratti nelle domande 1-3. Ogni numero è etichettato **[MISURATO]**, **[PROIETTATO]** o **[SIMULATO]** e associato alla rispettiva baseline. La colonna *Cosa copre / non copre di noi* esplicita il confronto col nostro caso (Zen 2 no VNNI, single-user, SSM, byte-level BPB, pshufb-LUT bit-exact 4.2-5× vs fp32, +0.018-0.022 BPB proiezioni ternarizzate). I dettagli per ogni lavoro sono nelle sezioni 1-3.

Tabella 1 — Lavori su kernel LUT CPU, quantizzazione SSM, QAT ternary: tecnica, baseline, numero misurato, hardware, regime, copertura del nostro caso.

Lavoro	Tecnica	Baseline	Numero misurato	Hardware	Regime	Cosa copre / non copre di noi
T-MAC [1]	LUT bit-wise su pesi low-bit (int1/2/3/4), pshufb AVX2 + NEON TBL	llama.cpp (dequant)	11.2× (1-bit), 5.8× (2-bit), 4.7× (3-bit), 3.1× (4-bit) single-thread GEMV [MISURATO]; 4× throughput headline, 70% energy	Apple M2-Ultra, Jetson Orin, Surface Book 3 (Ice Lake), RPi5, Snapdragon X Elite	Multi-thread: memory-bandwidth-bound; single-thread: compute-bound	COPRE: pshufb AVX2, ternary, single-user. NON COPRE: Zen 2 (README avverte old-gen x86), bit-exact (table quant), dReLU, SSM, BPB

Lavoro	Tecnica	Baseline	Numero misurato	Hardware	Regime	Cosa copre / non copre di noi
bitnet.cpp [2]	TL (element-wise LUT ternary) + I2_S (MAD-based 2-bit+scale), vpshufb AVX2 + vqtbl1q NEON	llama.cpp FP16 + TQ1_0/TQ2_0 + T-MAC	6.25× vs FP16 [MISURATO]; range x86 [2.37×, 6.17×]; 100B BitNet 5-7 tok/s	Apple M2 Ultra (ARM >800 GB/s), Intel i7-13700H (Raptor Lake AVX2+AVX-VNNI <100 GB/s)	Decode batch=1: memory-bandwidth-bound; prefill: compute-bound	COPRE: pshufb AVX2, ternary, bit-exact (TL1_1/TL2_1/I2_S), single-user. NON COPRE: Zen 2, dReLU, SSM, BPB
BitNet [3]	QAT binary {-1,+1} W1A8 (framing, non kernel)	FP16 Transformer	Scaling law match a 30B; energy reductions teorici	GPU training; inference non testato CPU	N/A (paper QAT, non kernel)	COPRE: framing QAT. NON COPRE: kernel CPU, pshufb, ternary, Zen 2, SSM, BPB, dReLU
LUT-NN [4]	LUT centroid-based (differentiable centroid learning)	related work (non fp32 diretto)	latency ≤6.8×, FLOPs ≤16×, model size ≤7× [MISURATO]	CPU Intel + ARM (non specificato)	On-device DNN (CNN, speech, NLP)	COPRE: LUT CPU, single-user edge. NON COPRE: pshufb bit-wise, ternary, bit-exact, Zen 2, LLM, SSM, BPB
ShiftAddLLM [5]	BCQ binary + LUT queries (shift+add), CUDA kernel	OPTQ, LUT-GEMM, QuIP, AWQ	-5.6 PPL (3-bit), -22.7 PPL (2-bit); latency reduction 6.5-60.1% [MISURATO]	NVIDIA A100-80GB GPU (PCIe)	GPU inference (non CPU)	COPRE: LUT su pesi binary con attivazioni high-precision (stesso principio). NON COPRE: CPU, pshufb, ternary, Zen 2, SSM, BPB
TernGEMM [6]	TernMAC add/sub su pesi ternari packed (no pshufb-LUT)	GEMMLowp (INT8)	1.68× vs GEMMLowp; ResNet50 2.052×, MobileNet-V2 2.973× [MISURATO]	CPU Intel + ARM (assembly, non specificato modello)	DNN inference (CNN), single-thread	COPRE: ternary CPU SIMD, single-user. NON COPRE: pshufb-LUT, Zen 2, LLM, SSM, BPB, dReLU

Lavoro	Tecnica	Baseline	Numero misurato	Hardware	Regime	Cosa copre / non copre di noi
DeepGEMM [7]	LUT-16 su W2A2, AVX2 256-bit, LUT in L1	QNNPACK INT8	1.74× vs INT8 QNNPACK [MISURATO] $\approx 7\times$ vs FP32 teorico	Intel i7-9700K (Coffee Lake 2018, AVX2 no AVX-512 no VNNI) — hardware più vicino a Zen 2	CNN inference, LUT in L1 $\rightarrow$ compute-bound	COPRE: x86 AVX2-only consumer no VNNI, LUT CPU, bit-exact, L1 cache, single-user. NON COPRE: ternary (W2A2), pshufb esplicito, LLM, SSM, BPB, dReLU
T-SAR [8]	Ternary-to-binary decomposition + LUT in-register, richiede ISA extension custom (TLUT/TGE MV)	Bitnet.cpp TL-2 + T-MAC	5.6-24.5× GEMM, 1.1-86.2× GEMV [SIMULATO gem5]; decode 6.4× Workstation, 4.1-4.2× Laptop/Mobile	gem5 sim su AMD Ryzen 9950X (Zen 5), 7840U (Zen 4), Intel N250	Decode: memory-bandwidth-bound (T-SAR rimuove TLUT traffic)	COPRE: ternary, AVX2 focus, single-user, bit-exact. NON COPRE: pshufb-LUT puro software (richiede ISA extension), Zen 2, silicio reale (gem5 sim), dReLU, SSM, BPB
TABv2 [9]	Bitwise GEMM con popcount SIMD (non pshufb-LUT)	baseline ternary/binary bitwise GEMM	Numeri in paper IEEE paywall [MISURATO]	CPU AVX2 (Intel) + ARM NEON; GPU CUDA separato	Edge inference, ternary/binary NN	COPRE: ternary AVX2 CPU. NON COPRE: pshufb-LUT (popcount-based), LLM, SSM, BPB, dReLU, Zen 2
Quamba [10]	PTQ W8A8 su Mamba-1; percentile clamping x + Hadamard y	FP16	PPL 9.45 $\rightarrow$ 9.91 (+0.46) Mamba-2.8B; -0.9% zero-shot 6-task; 1.72× speedup Orin Nano [MISURATO]	Nvidia A5000 GPU + Orin Nano edge GPU	GPU edge inference	COPRE: Mamba-1, SSM selettivo, PTQ pesi+attivazioni. NON COPRE: Mamba-2, ternary/W1, BPB, CPU, single-user, SWA, dReLU, pshufb

Lavoro	Tecnica	Baseline	Numero misurato	Hardware	Regime	Cosa copre / non copre di noi
Quamba2 [11]	PTQ W8A8/W4A8 /W4A16 + mixed-precision su Mamba-1+Mamba-2; sort-and-cluster x, per-state-group B/C	FP16	Mamba-2 8B W4A8 -1.4% zero-shot; W4A16 -0.6%; MMLU -5.8% (W4A8); 1.6% drop avg + 3× speedup gen [MISURATO]	Nvidia A5000 + Orin Nano edge GPU	GPU edge inference	COPRE: Mamba-1+Mamba-2, SSM selettivo, PTQ fino a W4, mixed-precision. NON COPRE: ternary/W1/W2, BPB, CPU, single-user, SWA, dReLU, pshufb, analisi per singola proiezione
KL-Lens [12]	PTQ + mixed-precision sensitivity (KL divergence forward-only)	FP16	Mamba-1.4B CPU: FP16=11.22 → INT4=60.55 → KL-MP p05=11.22 [MISURATO]; x_proj $\Delta=0.27$ (più sensibile), dt_proj $\Delta=3.6e-4$ (quasi insensibile)	Intel Lunar Lake CPU + GPU	CPU consumer x86 (Intel Lunar Lake)	COPRE: Mamba-1+Mamba-2, SSM, CPU consumer x86, mixed-precision, analisi per sottotipo proiezione. NON COPRE: ternary/W1 (max INT4), BPB, single-user esplicito, SWA, dReLU, pshufb
Mamba-PTQ [13]	PTQ W8/W4/W8A8 su Mamba-1; outlier channel analysis	non quantizzato (presunto FP16)	W8 (mlp) -0.5% LAMBADA; W4 (mlp) collasso LAMBADA a 0.02%; outlier removal -12.61% (130m), -17.49% (2.8b) [MISURATO]	non riportato (probabile GPU)	N/A (analisi accuracy, no profiling)	COPRE: Mamba-1, SSM, PTQ baselines proiezioni, analisi dt_proj NON-outlier. NON COPRE: Mamba-2, ternary, BPB, CPU, single-user, SWA, dReLU, pshufb, stato h/A/B/C/ Δ

Lavoro	Tecnica	Baseline	Numero misurato	Hardware	Regime	Cosa copre / non copre di noi
MambaQuant [14]	PTQ W8A8/W4A8 su Mamba-1; KLT-enhanced rotation + smooth-fused Hadamard	FP16	Mamba-2.8B W8A8 -0.1%; W4A8 -3.7%; <1% accuracy loss W8A8 [MISURATO]	GPU non specificata	GPU inference	COPRE: Mamba-1, SSM, PTQ W8A8/W4A8, analisi outlier per proiezione. NON COPRE: Mamba-2 (solo Vim vision), ternary, BPB, CPU, single-user, SWA, dReLU, pshufb
Q-Mamba [15]	PTQ W8A8H4 (include state cache H 4-bit) su Mamba-2	non dichiarato	Mamba-2 2.7B: 50% memory reduction + 2.13% accuracy degradation [MISURATO]	non dichiarato	N/A	COPRE: Mamba-2, SSM, quantizzazione e state cache (unico). NON COPRE: ternary, BPB, CPU, single-user, SWA, dReLU, pshufb, analisi per sottotipo proiezione
BitNet [3]	QAT binary {-1,+1} W1A8 da zero	FP16 Transformer	Scaling law match; gap cala con scala (125M-30B) [MISURATO]	GPU training	N/A (paper QAT)	COPRE: QAT da zero, Transformer LM, small scale <100M PARZIALE (125M). NON COPRE: ternary, 5M, BPB, dReLU, SSM, single-user CPU

Lavoro	Tecnica	Baseline	Numero misurato	Hardware	Regime	Cosa copre / non copre di noi
BitNet b1.58 [16]	QAT ternary {-1,0,+1} W1.58A8 da zero, LLaMA-style SwiGLU	FP16 LLaMA riprodotto	Match FP16 a 3B+; gap cala con scala (3B-70B) [MISURATO]	GPU training; inference FasterTransformer + Ladder 2-bit	N/A (paper QAT)	COPRE: ternary, QAT da zero, LLaMA-style SwiGLU (simile dReLU). NON COPRE: 5M (min 3B), <100M, BPB, dReLU esplicito, SSM, single-user CPU
BitNet v2 [17]	QAT ternary + 4-bit act (Hadamard), 400M-7B	BitNet b1.58 (riferimento ternary)	PPL C4 400M: b1.58=13.37 → v2(a4)=13.78 (+0.28); 7B: 9.09 → 9.24 (+0.15); gap cala con scala [MISURATO]	GPU training	N/A (paper QAT)	COPRE: ternary, QAT, scale <100M PARZIALE (400M, minore). NON COPRE: 5M, BPB, dReLU, SSM, single-user CPU
Spectra/ TriLM [18]	QAT ternary {-1,0,+1} da zero, LLaMA-style SwiGLU, 1 scale FP16 per matrix	FloatLM (FP16) stessa parameter count	Gap misurabile <1B, gap → 0 >1B commonsense; 3.9B match FloatLM [MISURATO]; 99M è scala min nella letteratura ternary-LM	GPU training (54 modelli × 300B tokens)	N/A (paper QAT)	COPRE: ternary, QAT da zero, gated MLP SwiGLU (affine dReLU), small scale <100M PARZIALE (99M, migliore). NON COPRE: 5M (min 99M, 20× più grande), BPB, dReLU esplicito, SSM, single-user CPU

Lavoro	Tecnica	Baseline	Numero misurato	Hardware	Regime	Cosa copre / non copre di noi
TernaryBERT [19]	PTQ + distillazione ternary {-1,0,+1} su BERT fine-tunato	FP32 BERT-base	GLUE avg 78.2 → 76.9 (W2A8, -1.3 pts); TernaryTinyBERT 14.5M ~pari [MISURATO]	GPU training (Huawei Ascend)	N/A (paper QAT+distill)	COPRE: ternary, small scale <100M PARZIALE (14.5M TernaryTinyBERT). NON COPRE: QAT da zero su LM (è PTQ+distill BERT classification), 5M, BPB, dReLU, SSM, single-user CPU
TWN [20]	QAT ternary {-1,0,+1} da zero su CNN, threshold-based + scaling	FP32 FPWN	ImageNet ResNet18 -3.6%, ResNet18B 1.5× -2.3%; gap cala con scala (11M → 16M) [MISURATO]	GPU training (4 GPU ImageNet)	N/A (paper QAT CNN)	COPRE: ternary, QAT da zero, small scale <100M SÌ (tutti <20M). NON COPRE: language modeling (è vision), Transformer, 5M, BPB, dReLU, SSM, single-user CPU
PT ² -LLM [21]	PTQ ternary ~1.58-bit su LLaMA-7B/13B/70B (NON QAT)	FP16	L-7B Wiki2 PPL 5.68 → 11.39 (~2× peggio); zero-shot avg 61.73 → 45.07 (-16.7 pts) [MISURATO]; gap NON cala con scala	GPU (LLaMA family)	N/A (paper PTQ)	COPRE: niente di rilevante per QAT ternary small scale. NON COPRE: QAT, small scale, pretraining, BPB. [CONTROLL O NEGATIVO]: conferma indiretta che trend 'gap cala con scala' è specifico QAT

Fonti: [1]-[23], vedi Bibliografia. [SIMULATO] = *gem5 simulation non silicio reale*.

5. Domanda 4 — Prior-art da citare (lista secca)

Lista secca di ~15 lavori che un reviewer considererebbe prior-art del nostro pshufb-LUT e del nostro "proiezioni precision-hungry", raggruppati per famiglia. Per ciascuno: 1 riga su cosa copre e cosa NON copre del nostro caso (Zen 2 senza VNNI, single-user, SSM, byte-level BPB, pshufb-LUT bit-exact 4.2-5× vs fp32, +0.018-0.022 BPB proiezioni ternarizzate).

5.1 Famiglia LUT-kernel su CPU (prior-art del nostro pshufb-LUT)

- **T-MAC [1]** (Microsoft, EuroSys 2025) — LUT bit-wise su pesi low-bit con pshufb AVX2 + NEON TBL. **COPRE**: pshufb-LUT AVX2 path, ternary, single-user. **NON COPRE**: Zen 2 (README avverte old-gen x86), bit-exact (usa table quantization), dReLU, SSM, BPB.
- **bitnet.cpp [2]** (Microsoft, ACL 2025) — TL (element-wise LUT ternary) + I2S (*MAD-based 2-bit+scale*), *vpshufb* AVX2 + *vqtbl1q* NEON, *lossless*. ****COPRE****: *pshufb* AVX2, ternary, *bit-exact* (TL11/TL21/I2S), single-user. **NON COPRE**: Zen 2 (Raptor Lake solo), dReLU, SSM, BPB.
- **DeepGEMM [7]** (Intel, CVPR 2023W) — LUT-16 su W2A2, AVX2 256-bit, LUT in L1. **COPRE**: x86 AVX2-only consumer no VNNI (Intel i7-9700K Coffee Lake, hardware più vicino a Zen 2), LUT CPU, bit-exact, L1 cache. **NON COPRE**: ternary (W2A2 uniform), pshufb esplicito, LLM, SSM, BPB, dReLU.
- **TernGEMM [6]** (Seoul Nat'l U., SiPS 2021) — TernMAC add/sub su pesi ternari packed (no pshufb-LUT). **COPRE**: ternary CPU SIMD, single-user. **NON COPRE**: pshufb-LUT (add/sub-based), Zen 2, LLM, SSM, BPB, dReLU.
- **LUT-NN [4]** (Microsoft, MobiCom 2023) — LUT centroid-based (differentiable centroid learning). **COPRE**: LUT CPU, single-user edge. **NON COPRE**: pshufb bit-wise (centroid-based), ternary, bit-exact, Zen 2, LLM, SSM, BPB.
- **T-SAR [8]** (UC Irvine, nov 2025) — Ternary-to-binary decomposition + LUT in-register, richiede ISA extension custom. **COPRE**: ternary, AVX2 focus, single-user, bit-exact. **NON COPRE**: pshufb-LUT puro software (richiede ISA extension non esistente su Zen 2), silicio reale (gem5 sim), dReLU, SSM, BPB.
- **TABv2 / FasterTNN [9]** (ETH Zurich, IEEE TVLSI 2026) — Bitwise GEMM con popcount SIMD (non pshufb-LUT). **COPRE**: ternary AVX2 CPU. **NON COPRE**: pshufb-LUT (popcount-based), LLM, SSM, BPB, dReLU, Zen 2.
- **ShiftAddLLM [5]** (Georgia Tech/Intel/Google, NeurIPS 2024) — BCQ binary + LUT queries (shift+add), CUDA kernel GPU. **COPRE**: LUT su pesi binary con attivazioni high-precision (stesso principio del nostro setup). **NON COPRE**: CPU, pshufb, ternary, Zen 2, SSM, BPB.

5.2 Famiglia SSM quantization (prior-art del nostro "proiezioni precision-hungry")

- **Quamba [10]** (IST Austria, ICLR 2025) — PTQ W8A8 su Mamba-1; percentile clamping $x + \text{Hadamard } y$. **COPRE:** Mamba-1, SSM selettivo, PTQ pesi+attivazioni. **NON COPRE:** Mamba-2, ternary/W1, BPB, CPU, single-user, SWA, dReLU, pshufb, analisi per singola proiezione.
- **Quamba2 [11]** (IST Austria, ICML 2025) — PTQ W8A8/W4A8/W4A16 + mixed-precision su Mamba-1+Mamba-2. **COPRE:** Mamba-1+Mamba-2, SSM, PTQ fino a W4, mixed-precision. **NON COPRE:** ternary/W1/W2, BPB, CPU, single-user, SWA, dReLU, pshufb, analisi per singola proiezione.
- **KL-Lens [12]** (UC San Diego, CVPR 2026W) — PTQ + mixed-precision sensitivity (KL divergence forward-only) su Mamba-1+Mamba-2, Intel Lunar Lake CPU. **COPRE:** Mamba-1+Mamba-2, SSM, CPU consumer x86, mixed-precision, **analisi per sottotipo proiezione (x_proj, dt_proj)** — unico paper con questa granularità. **NON COPRE:** ternary/W1 (max INT4), BPB, single-user esplicito, SWA, dReLU, pshufb. **[CONTRADDIZIONE PARZIALE]:** isola x_proj come colpevole primario.
- **Mamba-PTQ [13]** (Pierro & Abreu, arXiv lug 2024) — PTQ W8/W4/W8A8 su Mamba-1; outlier channel analysis. **COPRE:** Mamba-1, SSM, PTQ baselines proiezioni, **analisi dt_proj NON-outlier** (allineato col nostro finding). **NON COPRE:** Mamba-2, ternary, BPB, CPU, single-user, SWA, dReLU, pshufb, stato h/A/B/C/ Δ .
- **MambaQuant [14]** (Houmo AI + università cinesi, ICLR 2025) — PTQ W8A8/W4A8 su Mamba-1; KLT-enhanced rotation + smooth-fused Hadamard. **COPRE:** Mamba-1, SSM, PTQ W8A8/W4A8, analisi outlier per proiezione. **NON COPRE:** Mamba-2 (solo Vim vision), ternary, BPB, CPU, single-user, SWA, dReLU, pshufb.
- **Q-Mamba [15]** (ACL Findings 2025) — PTQ W8A8H4 (include state cache H 4-bit) su Mamba-2. **COPRE:** Mamba-2, SSM, quantizzazione state cache (unico). **NON COPRE:** ternary, BPB, CPU, single-user, SWA, dReLU, pshufb, analisi per sottotipo proiezione.

5.3 Famiglia QAT ternary small-scale (prior-art del nostro anchor 5M)

- **BitNet b1.58 [16]** (Microsoft, arXiv feb 2024) — QAT ternary $\{-1,0,+1\}$ W1.58A8 da zero, LLaMA-style SwiGLU. **COPRE:** ternary, QAT da zero, LLaMA-style SwiGLU (simile al nostro gated-dReLU). **NON COPRE:** 5M (min 3B), <100M, BPB, dReLU esplicito, SSM, single-user CPU.
- **Spectra/TriLM [18]** (Nolano Research/MILA, ICLR 2025 Spotlight) — QAT ternary $\{-1,0,+1\}$ da zero, LLaMA-style SwiGLU, 1 scale FP16 per matrix, 9 scale (99M-3.9B). **COPRE:** ternary, QAT da zero, gated MLP SwiGLU (affine dReLU), small scale <100M **PARZIALE** (99M, migliore della letteratura). **NON COPRE:** 5M (min 99M, 20× più grande), BPB, dReLU esplicito, SSM, single-user CPU. **Riferimento primario per estrapolazione del trend.**

- **BitNet v2 [17]** (Microsoft, arXiv giu 2025) — QAT ternary + 4-bit activation (Hadamard), 400M-7B. **COPRE**: ternary, QAT, scale <100M PARZIALE (400M, minore fra BitNet-family). **NON COPRE**: 5M, BPB, dReLU, SSM, single-user CPU.
- **BitNet [3]** (Microsoft, arXiv ott 2023, JMLR 2025) — QAT binary $\{-1,+1\}$ W1A8 da zero (framing fondativo). **COPRE**: QAT da zero, Transformer LM, small scale <100M PARZIALE (125M). **NON COPRE**: ternary (è binary), 5M, BPB, dReLU, SSM, single-user CPU.
- **TernaryBERT [19]** (Huawei, EMNLP 2020) — PTQ + distillazione ternary $\{-1,0,+1\}$ su BERT fine-tunato, 14.5M-109M. **COPRE**: ternary, small scale <100M PARZIALE (14.5M TernaryTinyBERT). **NON COPRE**: QAT da zero su LM (è PTQ+distill BERT classification), 5M, BPB, dReLU, SSM, single-user CPU.
- **TWN [20]** (Tsinghua/SJTU, arXiv 2016) — QAT ternary $\{-1,0,+1\}$ da zero su CNN, threshold-based + scaling, 50K-16M. **COPRE**: ternary, QAT da zero, small scale <100M SÌ (tutti <20M). **NON COPRE**: language modeling (è vision), Transformer, 5M, BPB, dReLU, SSM, single-user CPU. **Riferimento storico fondativo**.
- **PT²-LLM [21]** (SJTU/ETH/Tencent, arXiv set 2025) — PTQ ternary ~ 1.58 -bit su LLaMA-7B/13B/70B (NON QAT). **COPRE**: niente di rilevante per QAT ternary small scale. **NON COPRE**: QAT, small scale, pretraining, BPB. **[CONTROLLO NEGATIVO]**: conferma indiretta che trend "gap cala con scala" è specifico QAT, non PTQ.
- **BinaryBERT [23]** (Huawei, ACL 2021) — Binary $\{-1,+1\}$ BERT-base 109M con ternary weight splitting. **COPRE**: binary (non ternary), small scale <100M PARZIALE (109M). **NON COPRE**: ternary puro, 5M, BPB, dReLU, SSM, single-user CPU.

6. Contraddizioni e allineamenti con le nostre misure

Box espliciti per ciascuna contraddizione/allineamento tra le nostre misure e la letteratura. Ogni box riporta: il numero nostro, il numero del paper, flag **[CONTRADDIZIONE]** o **[ALLINEATO]**, e la fonte. **Le nostre misure**: (a) pshufb-LUT $4.2-5\times$ vs fp32 (bit-exact) sul MLP ternario su Zen 2 AVX2 no VNNI; (b) proiezioni SSM ternarizzate = $+0.018-0.022$ BPB (doppio braccio, misurato); (c) sensibilità NON isolata in x/dt ; (d) $+0.013\pm 0.005$ BPB combinato ternary+dReLU a 5M (anchor).

6.1 Contraddizioni

[CONTRADDIZIONE FORTE — KL-Lens su x_{proj}]. KL-Lens (CVPR 2026W, arXiv 2604.13440) isola esplicitamente x_{proj} come **colpevole primario** ($\Delta=0.27$, $>10\times$ gli altri sottotipi: $inproj$, $outproj$, $dtproj$, MoE up/down tutti ≤ 0.014). Il nostro finding dice "sensibilità NON isolata in x/dt " $\rightarrow$ ****contraddice sul lato x_{proj}** . Numero nostro: **sensibilità non isolata (nessun organo identificato come colpevole dominante)**. Numero paper: x_{proj} $\Delta=0.27$ vs ≤ 0.014 degli altri. Possibile spiegazione della divergenza: **KL-Lens usa Hymba hybrid SSM-Transformer-MoE (architettura diversa dal nostro Mamba-1 puro) e metrica ΔPPL su WikiText-2 (non BPB byte-level)**. Azione consigliata in review**: dichiarare esplicitamente la contraddizione e citare KL-Lens; motivare la divergenza con l'architettura (hybrid vs puro Mamba-1) e la metrica (PPL vs BPB).

[CONTRADDIZIONE TENDENZIALE — PT²-LLM su scala]. PT²-LLM (arXiv 2510.03267) mostra che PTQ ternary ha gap $\sim 2\times$ PPL su 7B-70B che **NON si chiude con scala**. Il nostro anchor ($+0.013\pm 0.005$ BPB a 5M) segue il trend QAT (gap cala con scala). **Numero nostro:** $+0.013$ BPB a 5M (QAT, gap piccolo ma >0 , consistente con trend *gap cala con scala*). **Numero paper:** $\sim 2\times$ PPL su 7B-70B (PTQ, gap NON cala con scala). **Conclusione:** non è una contraddizione diretta (il nostro è QAT, il paper è PTQ), ma **conferma indiretta** che il trend positivo è specifico del QAT. **Azione in review:** citare PT²-LLM come controllo negativo per mostrare che il trend QAT non è una proprietà generale della ternarizzazione.

6.2 Allineamenti

[ALLINEATO FORTE — Mamba-PTQ su dt_proj]. Mamba-PTQ (arXiv 2407.12397) esplicitamente dice che **dt_proj** mostra "**almost no outliers**" e non isola singolo colpevole fra in/x/out (tutte $<1\%$ canali outlier). **Numero nostro:** sensibilità NON isolata in x/dt. **Numero paper:** dtproj "**almost no outliers**"; in/x/out tutti con stesso pattern di outlier ($<1\%$ canali). ****Allineato perfettamente**** sul lato dtproj.

[ALLINEATO PARZIALE — MambaQuant su x/dt]. MambaQuant (ICLR 2025) **NON isola x_proj o dt_proj come colpevoli**; identifica invece gateproj + outproj + matmul come i contenitori di outlier. xproj e dtproj hanno valori modesti in Fig.13 ($15.1/12.0/11.1$ e $14.9/13.6/4.1$ vs outproj $1371.6/1064.3/930.3$). ****Numero nostro**:** sensibilità NON isolata in x/dt. ****Numero paper**:** xproj/dt_proj NON identificati come colpevoli. **Allineato** con il nostro finding.

[ALLINEATO — KL-Lens su dt_proj]. KL-Lens (CVPR 2026W) isola anche **dt_proj come quasi-insensibile** ($\Delta=3.6\times 10^{-4}$). **Numero nostro:** sensibilità NON isolata in x/dt. **Numero paper:** dtproj $\Delta=3.6\times 10^{-4} \rightarrow$ "suggesting that extremely low-precision representations are feasible". ****Allineato**** sul lato dtproj (entrambi dicono che dtproj non è sensibile), anche se contraddice sul lato xproj (vedi 6.1).

[ALLINEATO — bitnet.cpp su speedup pshufb-LUT]. bitnet.cpp (ACL 2025) testa pshufb-LUT ternary lossless su Raptor Lake (con AVX-VNNI). Range x86 dichiarato [$2.37\times$, $6.17\times$] vs FP16. **Numero nostro:** $4.2-5\times$ vs fp32 su Zen 2 (no VNNI). **Numero paper:** [$2.37\times$, $6.17\times$] vs FP16 su Raptor Lake (con AVX-VNNI). **Allineato** — il nostro valore $4.2-5\times$ cade nel range dichiarato, anche se su hardware più vecchio (Zen 2 2019 vs Raptor Lake 2023). **Azione in review:** citare bitnet.cpp come prior-art diretto del pshufb-LUT ternary lossless; dichiarare che il nostro $4.2-5\times$ su Zen 2 è evidenza diretta non coperta dal paper (che non testa Zen 2).

[ALLINEATO — DeepGEMM su fattibilità AVX2-only]. DeepGEMM (CVPR 2023W) testa LUT-based GEMM su Intel i7-9700K Coffee Lake (AVX2-only, no AVX-512, no VNNI) — hardware architetturalmente simile al nostro Zen 2. **Numero nostro:** $4.2-5\times$ vs fp32 su Zen 2 AVX2-only no VNNI. **Numero paper:** $1.74\times$ vs INT8 QNNPACK su i7-9700K $\approx 7\times$ vs FP32 teorico (su W2A2 CNN, non LLM). **Allineato** — convertito, il range è coerente o superiore al nostro. **Azione in review:** citare DeepGEMM come evidenza di fattibilità su hardware AVX2-only consumer no VNNI; dichiarare che è l'hardware più vicino al nostro Zen 2 nella letteratura.

[ALLINEATO — Trend QAT ternary su anchor 5M]. Spectra/TriLM (ICLR 2025 Spotlight) mostra gap misurabile sotto 1B, gap $\rightarrow 0$ sopra 1B per commonsense/knowledge. BitNet b1.58: match FP16 a 3B+. BitNet (originale): *"gap becomes smaller as the model size grows"* (125M-30B). BitNet v2: gap 4-bit vs 8-bit activation cala da 400M a 7B. TWN (CNN): gap ResNet18 $\rightarrow$ ResNet18B cala da -3.6% a -2.3% . **Numero nostro:** $+0.013 \pm 0.005$ BPB combinato ternary+dReLU a 5M (vs fp32). **Numero paper:** gap piccolo ma >0 sotto 99M (Spectra/TriLM è il minimo), gap $\rightarrow 0$ sopra 1B. **Allineato** come direzione del trend (gap piccolo ma >0 sotto 99M, consistente con $+0.013$ BPB a 5M per estrapolazione). **Caveat:** validazione diretta impossibile — nessun paper ternary QAT scende sotto 99M o riporta BPB. **Azione in review:** citare Spectra/TriLM come riferimento primario per estrapolazione del trend; dichiarare il gap di evidenza a 5M.

6.3 Sintesi — contraddizioni da dichiarare in review

Contraddizioni da dichiarare esplicitamente in review (in ordine di severità):

- **[CONTRADDIZIONE FORTE] KL-Lens su x_proj:** il paper isola x_proj come colpevole primario ($\Delta=0.27$, $>10\times$ gli altri); il nostro finding dice "non isolata in x/dt". Da dichiarare e motivare con architettura (Hymba hybrid vs Mamba-1 puro) e metrica (PPL vs BPB).
- **[CONTRADDIZIONE TENDENZIALE] PT²-LLM su scala:** PTQ ternary non segue il trend *gap cala con scala* (gap $\sim 2\times$ PPL su 7B-70B). Il nostro anchor QAT è allineato col trend QAT. Da citare come controllo negativo per mostrare che il trend è specifico QAT.

Allineamenti da citare a supporto (in ordine di forza):

- **[ALLINEATO FORTE] Mamba-PTQ su dt_proj:** *"almost no outliers"* — conferma diretta che dt_proj non è sensibile.
- **[ALLINEATO PARZIALE] MambaQuant su x/dt:** non isola xproj/dtproj come colpevoli; identifica gateproj + outproj + matmul.
- **[ALLINEATO] KL-Lens su dt_proj:** $\Delta=3.6 \times 10^{-4}$ = quasi insensibile (conferma il nostro finding sul lato dtproj, anche se contraddice sul lato xproj).
- **[ALLINEATO] bitnet.cpp su speedup pshufb-LUT:** range x86 [$2.37\times$, $6.17\times$] contiene il nostro 4.2-5 $\times$.
- **[ALLINEATO] DeepGEMM su fattibilità AVX2-only:** $1.74\times$ vs INT8 su i7-9700K Coffee Lake (hardware più vicino a Zen 2) $\approx 7\times$ vs FP32 teorico.
- **[ALLINEATO] Trend QAT ternary su anchor 5M:** Spectra/TriLM, BitNet b1.58, BitNet v2, TWN tutti mostrano gap cala con scala; $+0.013$ BPB a 5M è consistente per estrapolazione.

Conclusione fattuale. Nessuna contraddizione formale con i nostri numeri di speedup (4.2-5 $\times$ vs fp32) o di costo BPB ($+0.013 \pm 0.005$ a 5M, $+0.018-0.022$ proiezioni ternarizzate). **Unica contraddizione sostanziale:** KL-Lens su x_proj (isola come colpevole primario, noi non lo isoliamo) — da dichiarare esplicitamente in review e motivare con architettura + metrica. **Zona bianca confermata:** nessun paper testa combinazione esatta del nostro setup (Zen 2 AVX2 no VNNI + pshufb-LUT ternary bit-exact + single-user + MLP gated-dReLU + SSM + byte-level BPB + 5M scala). Il nostro setup è in territorio non coperto; la letteratura supporta la fattibilità tecnica (pshufb-

LUT AVX2 path standard, AVX2-only consumer funziona, trend QAT coerente) ma non fornisce numeri diretti su Zen 2 + SSM + 5M.

7. Bibliografia

Bibliografia numerata. Formato: [N] Autori (Anno). Titolo. Venue/arXiv-id. URL. Le citazioni inline nel testo si riferiscono a questi numeri.

- [1] Wei, J.; Cao, S.; Cao, T.; Ma, L.; Wang, L.; Zhang, Y.; Yang, M. (USTC; Microsoft Research; UCAS) (2025). *T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge*. EuroSys 2025; arXiv:2407.00088. URL: <https://arxiv.org/abs/2407.00088> — repo: <https://github.com/microsoft/T-MAC>.
- [2] Mo, Z.; Wang, L.; Wei, J.; Zeng, Z.; Cao, S.; Ma, L.; Jing, N.; Cao, T.; Xue, J.; Yang, F.; Yang, M. (SJTU; Microsoft Research; USTC) (2025). *Bitnet.cpp: Efficient Edge Inference for Ternary LLMs*. ACL 2025 (Long); arXiv:2502.11880. URL: <https://arxiv.org/abs/2502.11880> — repo: <https://github.com/microsoft/BitNet>.
- [3] Wang, H.; Ma, S.; Dong, L.; Huang, S.; Wang, H.; Ma, L.; Yang, F.; Wang, R.; Wu, Y.; Wei, F. (Microsoft Research + UWashington + USTC + CAS) (2023/2025). *BitNet: Scaling 1-bit Transformers for Large Language Models*. arXiv:2310.11453; JMLR vol. 26 (24-2050), 2025. URL: <https://arxiv.org/abs/2310.11453>.
- [4] Tang, X.; Wang, Y.; Cao, T.; Zhang, L.L.; Chen, Q.; Cai, D.; Liu, Y.; Yang, M. (Microsoft Research + Zhejiang U.) (2023). *LUT-NN: Empower Efficient Neural Network Inference with Centroid Learning and Table Lookup*. MobiCom 2023; arXiv:2302.03213. URL: <https://arxiv.org/abs/2302.03213>.
- [5] (Georgia Tech; Intel Labs; Google; Google DeepMind) (2024). *ShiftAddLLM: Accelerating Pretrained LLMs via Post-Training Multiplication-Less Reparameterization*. NeurIPS 2024; arXiv:2406.05981. URL: <https://arxiv.org/abs/2406.05981> — repo: <https://github.com/GATECH-EIC/ShiftAddLLM>.
- [6] Choi, S.; Shim, K.; Choi, J.; Sung, W.; Shim, B. (Seoul National University) (2021). *TernGEMM: General Matrix Multiply Library with Ternary Weights for Fast DNN Inference*. IEEE SiPS 2021. DOI: <https://ieeexplore.ieee.org/document/9605039> — repo: <https://github.com/aiha-lab/TernGEMM>.
- [7] Ganji, D.C.; Ashfaq, S.; Saboori, E.; Sah, S.; Mitra, S.; AskariHemmat, M.; Hoffman, A.; Hassanien, A.; Léonardon, M. (Intel; IIT Hyderabad; altri) (2023). *DeepGEMM: Accelerated Ultra Low-Precision Inference on CPU Architectures Using Lookup Tables*. CVPR 2023 Workshops, pp. 4656-4664. URL: https://openaccess.thecvf.com/content/CVPR2023W/ECV/html/GanjiDeepGEMMAcceleratedUltraLow-PrecisionInferenceonCPUArchitecturesUsingLookupCVPRW2023_paper.html — repo: <https://github.com/Emily0330/DeepGEMM-paper-reproduce>.
- [8] Oh, H.; Nam, K.; Bhattacharjya, R.; Chen, H.; Das, T.; Yun, S.; Jang, S.; Ding, A.; Dutt, N.; Imani, M. (UC Irvine) (2025). *T-SAR: A Full-Stack Co-design for CPU-Only Ternary LLM Inference via In-Place SIMD ALU Reorganization*. arXiv:2511.13676, novembre 2025. URL: <https://arxiv.org/html/2511.13676v1>.

- [9] Fu, G.; Fischer, O.; Zhu, S.; Alonso, G. (ETH Zurich, FPGA Systems Group) (2026). *TABv2: A Faster Ternary and Binary Neural Network Inference Library on the Edge*. IEEE TVLSI 2026 (DOI: 10.1109/TVLSI.2025.3650684). Precursore: ISLPED 2025. URL: <https://ieeexplore.ieee.org/document/11360778> — repo: <https://github.com/fpgasystems/FasterTNN>.
- [10] Chiang, H.-Y.; Chang, C.; Frumkin, N.; Wu, K.-C.; Marculescu, D. (IST Austria; UT Austin; NYCU) (2025). *Quamba: A Post-Training Quantization Recipe for Selective State Space Models*. ICLR 2025; arXiv:2410.13229. URL: <https://arxiv.org/abs/2410.13229> — codice: <https://github.com/enyac-group/Quamba>.
- [11] Chiang, H.-Y.; Chang, C.; Frumkin, N.; Wu, K.-C.; Abdelfattah, M.S.; Marculescu, D. (IST Austria; UT Austin; NYCU) (2025). *Quamba2: A Robust and Scalable Post-training Quantization Framework for Selective State Space Models*. ICML 2025 (PMLR v267); arXiv:2503.22879. URL: <https://arxiv.org/abs/2503.22879> — codice: <https://github.com/enyac-group/Quamba>.
- [12] Kong, J.; Pandey, N.P.; Ponzina, F.; Rosing, T. (UC San Diego; SDSU) (2026). *A KL Lens on Quantization: Fast, Forward-Only Sensitivity for Mixed-Precision SSM–Transformer Models*. CVPR 2026 Workshop EDGE; arXiv:2604.13440. URL: <https://arxiv.org/abs/2604.13440> — codice: <https://github.com/jasonkongie/kl-ssm-quant>.
- [13] Pierro, A.; Abreu, S. (2024). *Mamba-PTQ: Outlier Channels in Recurrent Large Language Models*. arXiv:2407.12397, luglio 2024. URL: <https://arxiv.org/abs/2407.12397>.
- [14] Xu, T.; Liu, B.; Chen, Z.; Zhang, S.; et al. (Houmo AI + università cinesi) (2025). *MambaQuant: Quantizing the Mamba Family with Variance Aligned Rotation Methods*. ICLR 2025; arXiv:2501.13484. URL: <https://arxiv.org/abs/2501.13484>.
- [15] Chen, T.; Chen, Y.; Wang, P.; Xu, W.; Zhu, Z.; Cheng, J. (2025). *Q-Mamba: Towards more efficient Mamba models via post-training quantization*. Findings of ACL 2025, pp. 10594–10610. DOI: 10.18653/v1/2025.findings-acl.551. URL: <https://aclanthology.org/2025.findings-acl.551/>.
- [16] Ma, S.; Wang, H.; Ma, L.; Wang, L.; Wang, W.; Huang, S.; Dong, L.; Wang, R.; Xue, J.; Wei, F. (Microsoft Research + UChinese Academy of Sciences) (2024). *The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits*. arXiv:2402.17764, 27 febbraio 2024. URL: <https://arxiv.org/abs/2402.17764>.
- [17] Wang, H.; Ma, S.; Wei, F. (Microsoft Research + UChinese Academy of Sciences) (2025). *BitNet v2: Native 4-bit Activations with Hadamard Transformation for 1-bit LLMs*. arXiv:2504.18415, 13 giugno 2025. URL: <https://arxiv.org/abs/2504.18415>.
- [18] Kaushal, A.; Vaidya, T.; Mondal, A.K.; Pandey, T.; Bhagat, A.; Rish, I. (Nolano Research / MILA) (2025). *Spectra: Surprising Effectiveness of Pretraining Ternary Language Models at Scale*. arXiv:2407.12327, ICLR 2025 Spotlight. URL: <https://arxiv.org/abs/2407.12327>.
- [19] Zhang, W.; Hou, L.; Yin, Y.; Shang, L.; Chen, X.; Jiang, X.; Liu, Q. (Huawei Noah's Ark Lab) (2020). *TernaryBERT: Distillation-aware Ultra-low Bit BERT*. EMNLP 2020 (Findings); arXiv:2009.12812. URL: <https://arxiv.org/abs/2009.12812>.
- [20] Li, F.; Liu, B.; Wang, X.; Zhang, B.; Yan, J. (Tsinghua / SJTU / NUDT) (2016). *Ternary Weight Networks*. arXiv:1605.04711. URL: <https://arxiv.org/abs/1605.04711>.

- [21] Yan, X.; Bao, C.; Li, Z.; Zhang, T.; Yang, K.; Qin, H.; Xie, R.; Sun, X.; Zhang, Y. (SJTU + ETH Zürich + Tencent Hunyuan) (2025). *PT²-LLM: Post-Training Ternarization for Large Language Models*. arXiv:2510.03267, 27 settembre 2025. URL: <https://arxiv.org/abs/2510.03267>.
- [22] Liu, J.; Xiao, G.; Li, K.; Lee, J.D.; Han, S.; Dao, T.; Cai, T. (MIT + Princeton + NVIDIA + Together AI) (2024). *BitDelta: Your Fine-Tune May Only Be Worth One Bit*. arXiv:2402.10193. URL: <https://arxiv.org/abs/2402.10193>.
- [23] Bai, H.; Zhang, W.; Hou, L.; Shang, L.; Jin, J.; Jiang, X.; Liu, Q.; Lyu, M.; King, I. (Huawei Noah's Ark Lab + CUHK) (2021). *BinaryBERT: Pushing the Limit of BERT Quantization*. ACL 2021; arXiv:2012.15701. URL: <https://arxiv.org/abs/2012.15701>.